

数据挖掘编程助手

2026年3月19日

CodeGeeX

CodeGeeX 智能编程助手

每天帮助程序员编写500万行代码

视频介绍 

下载插件

代码生成与补全

注释生成

代码翻译

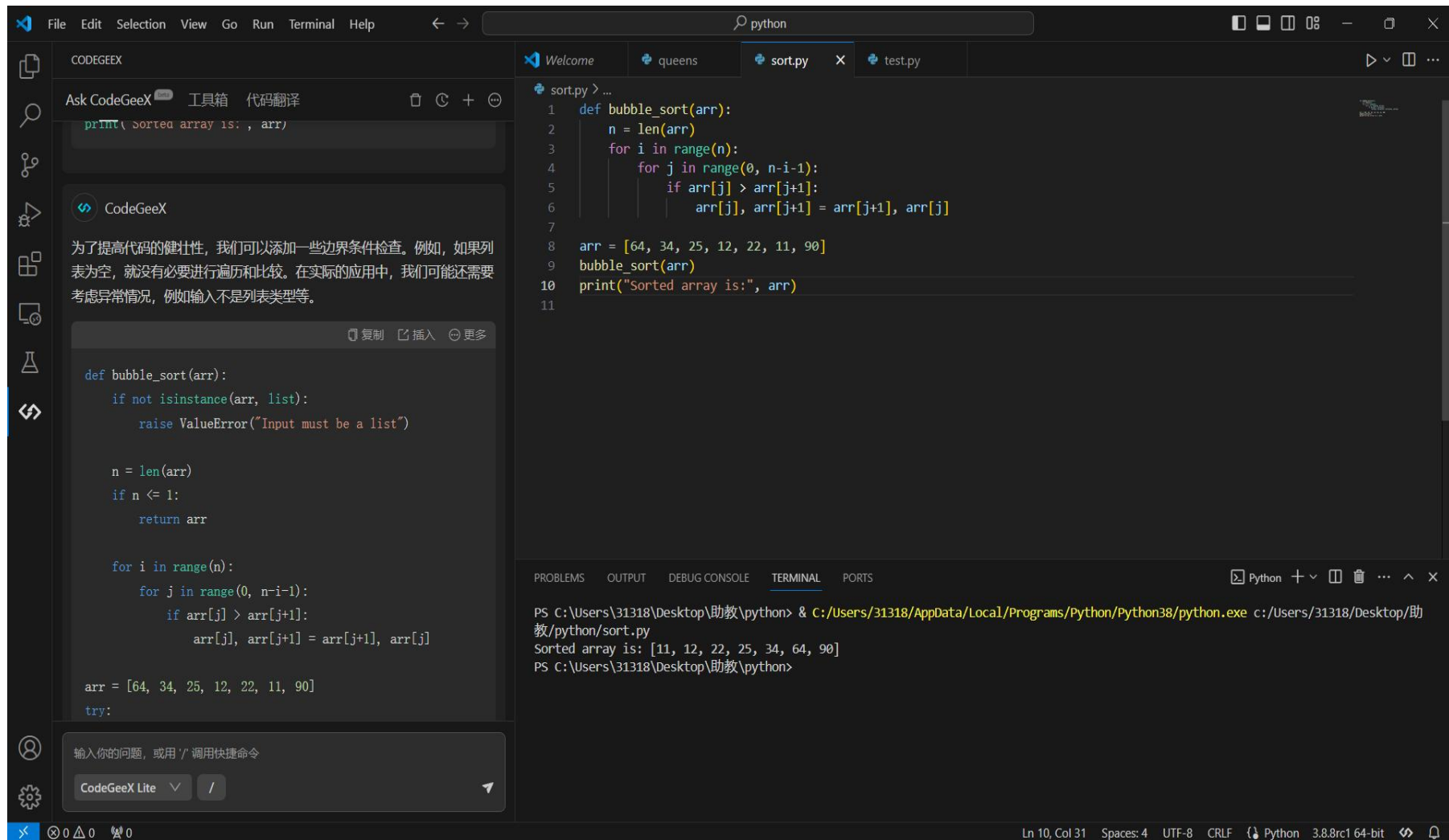
智能问答

```
1 import requests
2 # 请求 https://www.codegeex.cn/ 网站, 获取返回响应状态、响应头、响应内容, 并输出
3
4 response = requests.get('https://www.codegeex.cn/')
5 print(response.status_code)
6 print(response.headers)
7 print(response.text)
```

 CodeGeeX

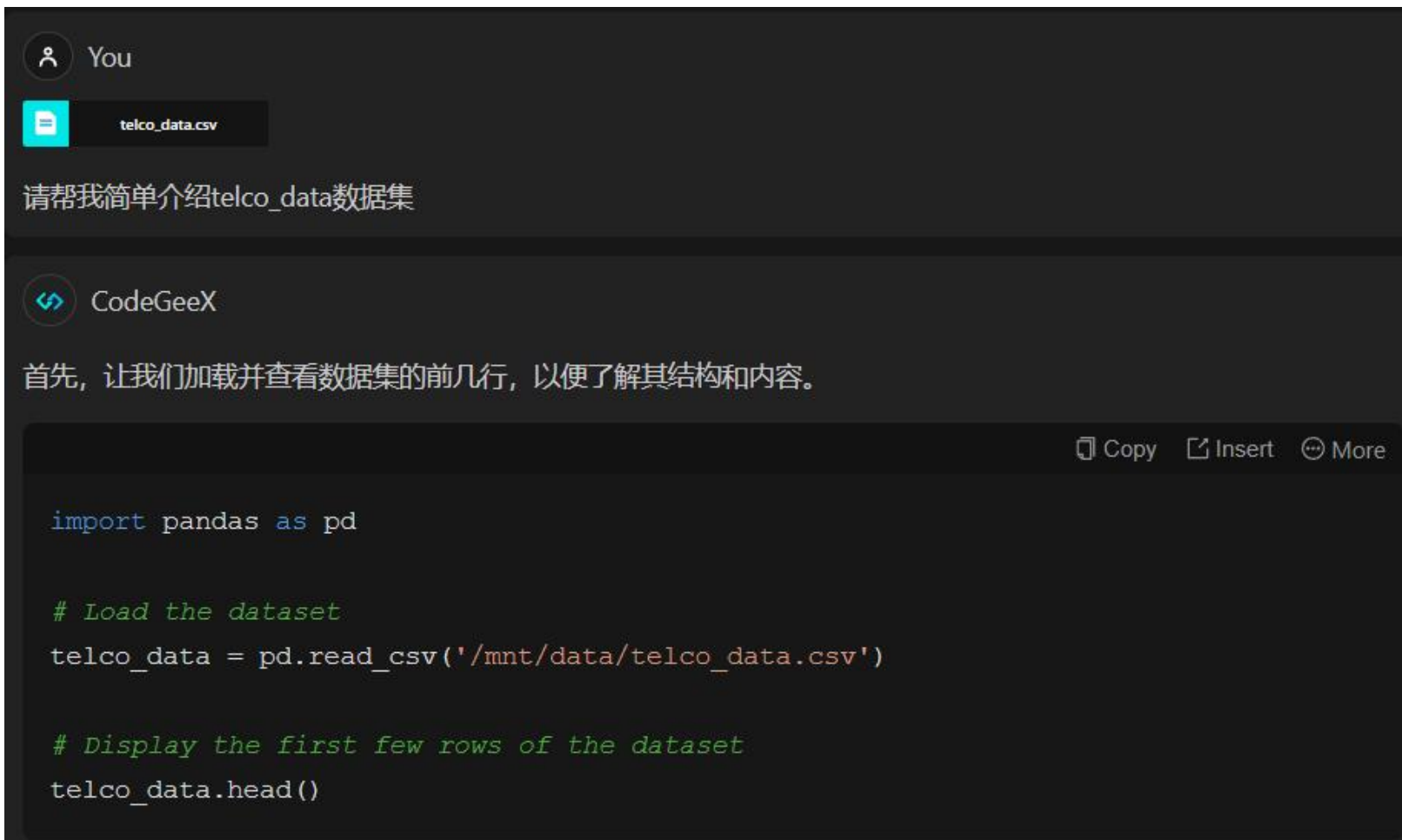
<https://codegeex.cn/>

CodeGeeX



CodeGeeX

实现数据分析



The screenshot shows the CodeGeeX interface. At the top, a user profile icon and the name 'You' are visible. Below this, a file icon and the name 'telco_data.csv' are shown. The user's prompt is: '请帮我简单介绍telco_data数据集'. The CodeGeeX response is: '首先, 让我们加载并查看数据集的前几行, 以便了解其结构和内容。'. Below the response, there is a code editor with a dark background and light-colored text. The code is a Python snippet that imports pandas, loads the dataset from a local path, and displays the first few rows. To the right of the code editor, there are icons for 'Copy', 'Insert', and 'More'.

```
import pandas as pd

# Load the dataset
telco_data = pd.read_csv('/mnt/data/telco_data.csv')

# Display the first few rows of the dataset
telco_data.head()
```

CodeGeeX

```

Copy Insert More
customerID gender SeniorCitizen Partner Dependents tenure PhoneService \
0 7590-VHVEG Female 0 Yes No 1 No
1 5575-GNVDE Male 0 No No 34 Yes
2 3668-QPYBK Male 0 No No 2 Yes
3 7795-CFOCW Male 0 No No 45 No
4 9237-HQITU Female 0 No No 2 Yes

MultipleLines InternetService OnlineSecurity ... DeviceProtection \
0 No phone service DSL No ... No
1 No DSL Yes ... Yes
2 No DSL Yes ... No
3 No phone service DSL Yes ... Yes
4 No Fiber optic No ... No

TechSupport StreamingTV StreamingMovies Contract PaperlessBilling \
0 No No No Month-to-month Yes
1 No No No One year No
2 No No No Month-to-month Yes
3 Yes No No One year No
4 No No No Month-to-month Yes

PaymentMethod MonthlyCharges TotalCharges Churn
0 Electronic check 29.85 29.85 No
1 Mailed check 56.95 1889.5 No
2 Mailed check 53.85 108.15 Yes
3 Bank transfer (automatic) 42.30 1840.75 No
4 Electronic check 70.70 151.65 Yes

[5 rows x 21 columns]
```

telco data 数据集包含以下列:

1. **customerID**: 客户的唯一标识符。
2. **gender**: 客户的性别。
3. **SeniorCitizen**: 客户是否是老年人。
4. **Partner**: 客户是否有伴侣。
5. **Dependents**: 客户是否有依赖项。
6. **tenure**: 客户与公司的关系时间 (以月为单位)。
7. **PhoneService**: 客户是否拥有电话服务。
8. **MultipleLines**: 客户是否拥有多条线路。
9. **InternetService**: 客户的互联网服务类型。
10. **OnlineSecurity**: 客户是否使用在线安全服务。
11. **DeviceProtection**: 客户是否使用设备保护服务。
12. **TechSupport**: 客户是否使用技术支持服务。
13. **StreamingTV**: 客户是否使用流媒体电视服务。
14. **StreamingMovies**: 客户是否使用流媒体电影服务。
15. **Contract**: 客户的合同类型。
16. **PaperlessBilling**: 客户是否使用无纸化账单。
17. **PaymentMethod**: 客户的支付方式。
18. **MonthlyCharges**: 客户的月度费用。
19. **TotalCharges**: 客户的总费用。
20. **Churn**: 客户是否已流失。

这个数据集可能用于预测客户是否会流失，或者用于分析哪些因素与客户流失有关。

CodeGeeX

查询缺失值

You

请帮我查询telco_data数据集数据缺失情况

CodeGeeX

```
# Checking for missing values in the dataset
missing_values = telco_data.isnull().sum()

missing_values
```

```
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
```

数据集中没有缺失值。每一列都是完整的。

CodeGeeX

进一步分析



CodeGeeX

You

性别在客户群体中的分布

CodeGeeX

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set the style of the visualization
sns.set(style="whitegrid")

# Create a count plot for the 'gender' column
plt.figure(figsize=(8, 6))
sns.countplot(data=telco_data, x='gender')
plt.title('Gender Distribution in Customers')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.show()
```

```
```result
【image】
```

上图显示了数据集中性别分布的情况。



```
import matplotlib.pyplot as plt
import seaborn as sns

Set the style of the visualization
sns.set(style="whitegrid")

Create a count plot for the 'gender' column
plt.figure(figsize=(8, 6))
sns.countplot(data=telco_data, x='gender')
plt.title('Gender Distribution in Customers')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.show()
```

[4] ✓ 1.2s

Python





# CodeGeeX

You

电话服务和互联网服务的使用情况，只需给出画图代码，不要运行

以下是绘制电话服务和互联网服务使用情况的代码：

```
import matplotlib.pyplot as plt
import seaborn as sns

Set the style of the visualization
sns.set(style="whitegrid")

Create a bar plot for the 'Partner' and 'Dependents' columns
plt.figure(figsize=(10, 7))
sns.barplot(data=telco_data, x='Partner', y='Dependents', palette='pastel')
plt.title('Usage of Telephone and Internet Services')
plt.xlabel('Partner')
plt.ylabel('Dependents')
plt.show()
```

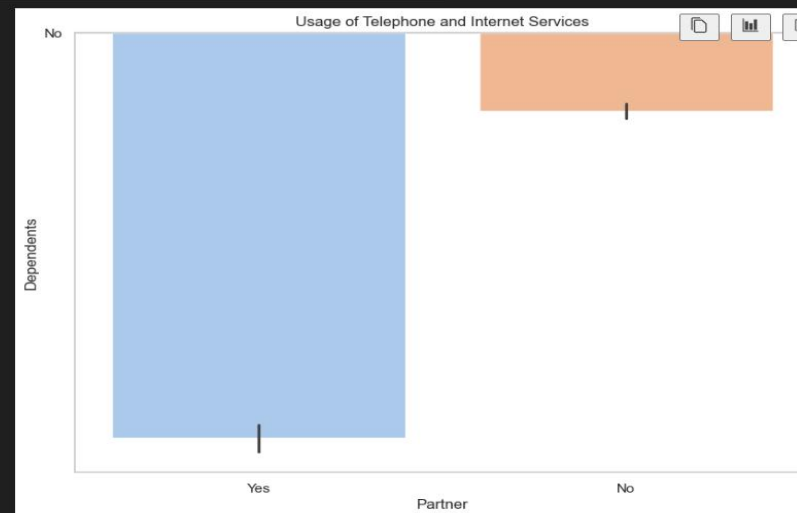
您可以使用上述代码在您自己的环境中运行它。

```
Create a bar plot for the 'Partner' and 'Dependents' columns
plt.figure(figsize=(10, 7))
sns.barplot(data=telco_data, x='Partner', y='Dependents', palette='pastel')
plt.title('Usage of Telephone and Internet Services')
plt.xlabel('Partner')
plt.ylabel('Dependents')
plt.show()
```

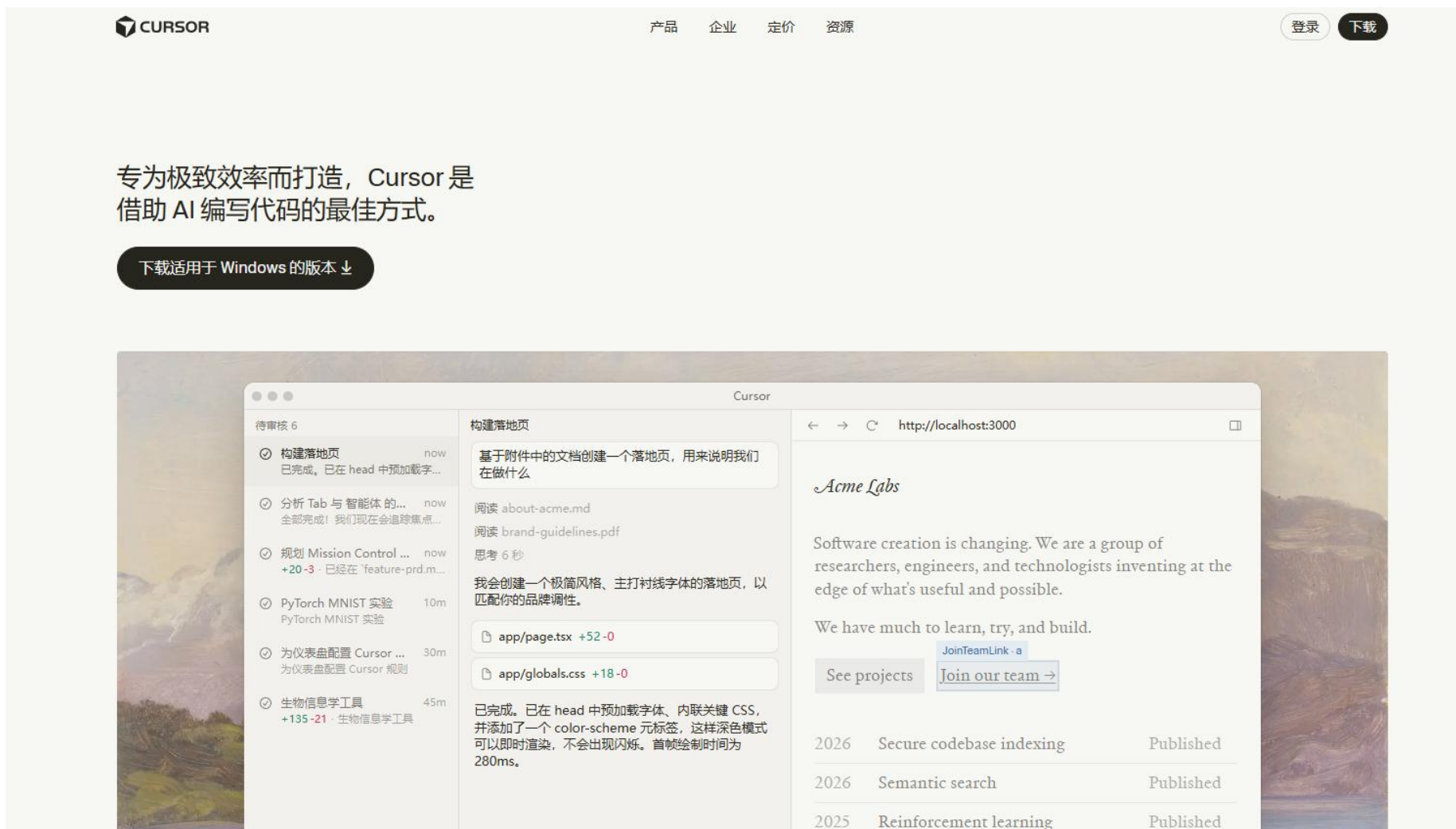
[5] ✓ 0.2s

Python

... C:\Users\Administrator\AppData\Local\Temp\ipykernel\_14960\3950625204.py:9: FutureWarning  
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.  
sns.barplot(data=telco\_data, x='Partner', y='Dependents', palette='pastel')



# Cursor (IDE)



# Cursor (IDE)

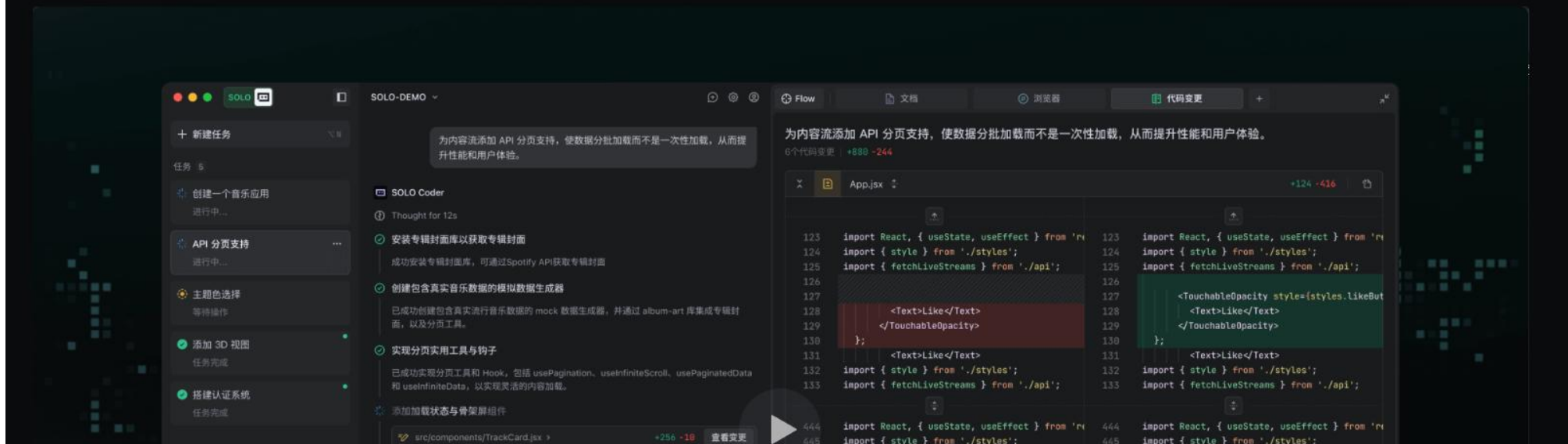
- Cursor Tab (预测性交互): 极速的无缝补全, 不仅能跨行生成代码, 还能精准预测你的“下一步修改位置 (Jumps)”。
- Composer 协同调度: 支持自然语言呼出对话框, AI 能够同时评估并修改项目中多个关联文件, 一键应用 (Accept All) 。
- Agent 自主模式: 具备高级读写和终端执行权限, 能自主查阅文档、运行环境、读取报错并自我修复。

# Trae (IDE)

## 和 TRAE 一起开启 SOLO 模式

TRAE SOLO 是一种高度自动化的开发方式，以 AI 为主导，可理解目标、承接上下文并调度工具，独立推进各阶段开发任务

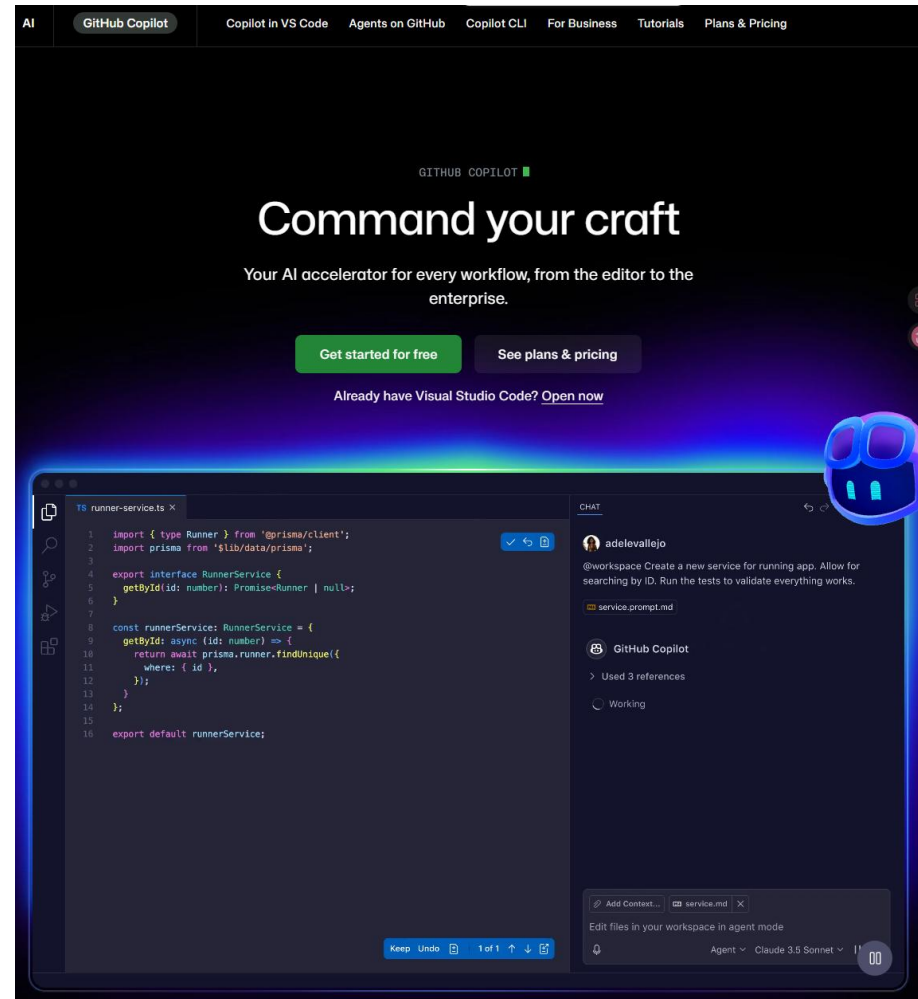
即刻探索 SOLO



# Trae (IDE)

- 双重开发模式：支持“IDE 模式”（保留开发者极强的控制感）与“SOLO 模式”（让 AI 完全主导任务，自动拆解步骤并推进开发）。
- 生态拓展 (MCP 协议)：支持无缝挂载外部工具、进行联网实时搜索以及上传庞大的自定义文档集。
- Trae Rules 个性化定制：允许开发者深度配置专属的 AI 工作规则，让代码生成完全贴合团队或个人的特定规范。

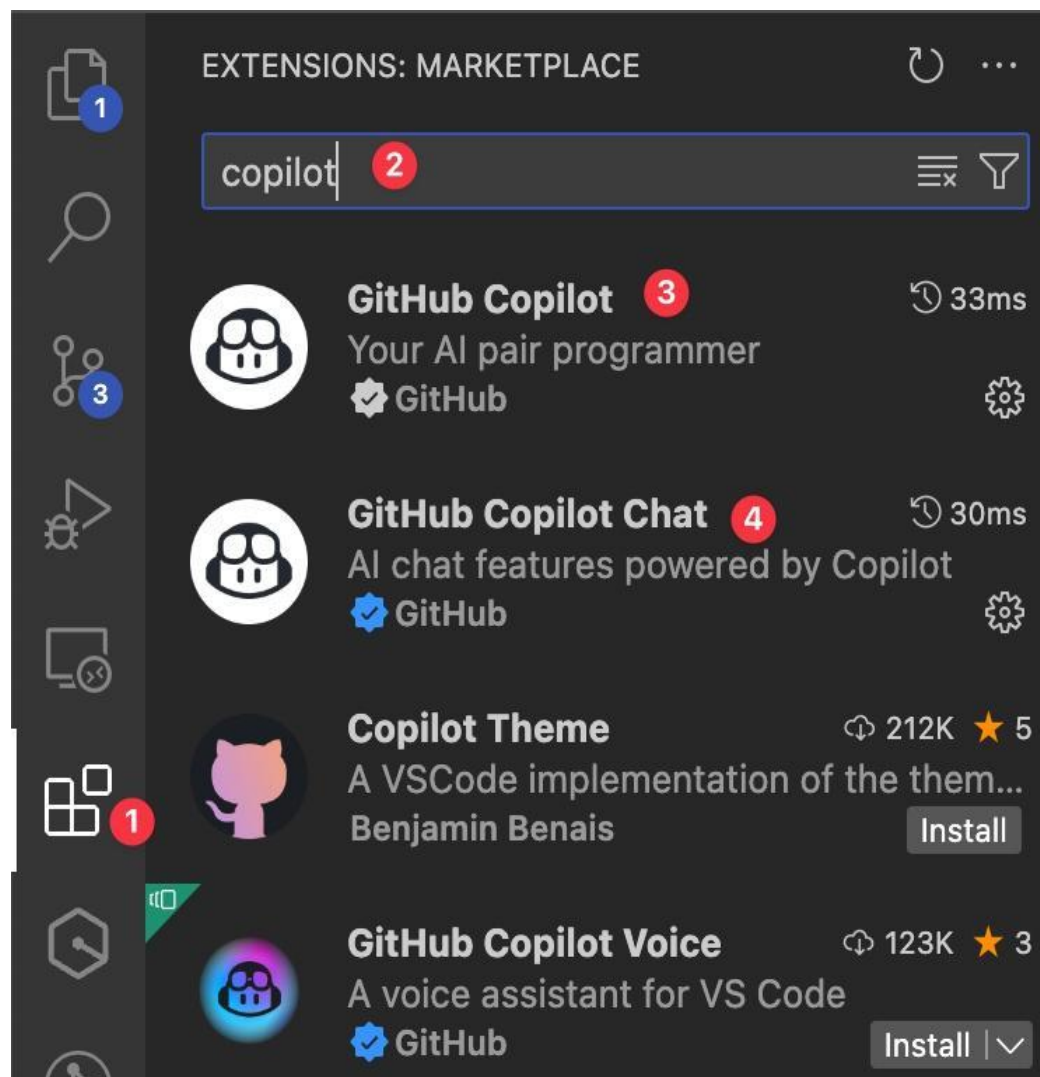
# GitHub Copilot (Vscode 插件)



# GitHub Copilot (Vscode 插件)

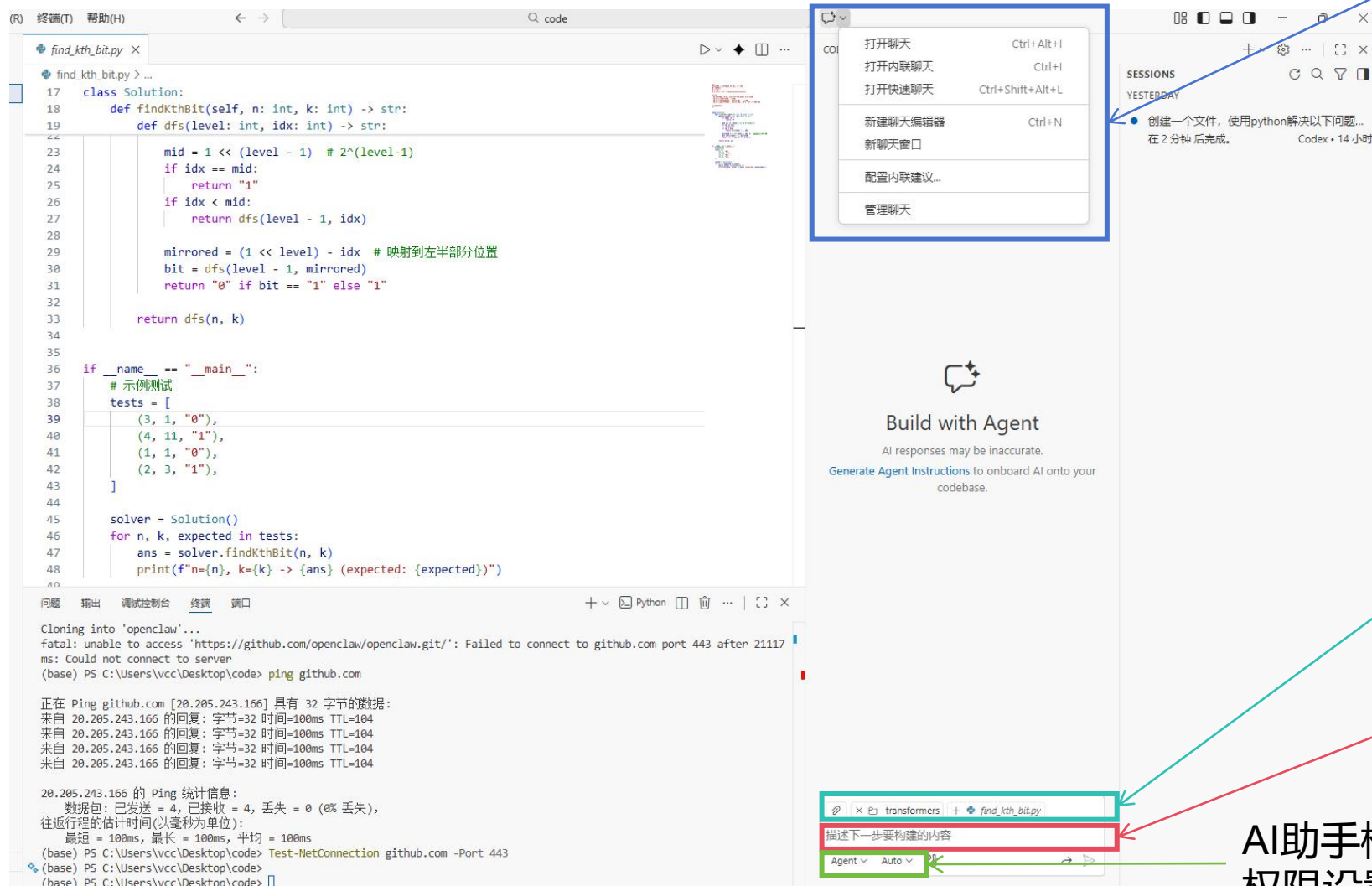
- 多模型接入 (Multi-Model): 不再只绑定单一模型, 2026版已支持开发者自由切换 OpenAI (GPT-4o/o3)、Anthropic (Claude 3.5 Sonnet/Opus) 甚至 Google 的大模型。
- Agent Mode (代理模式): 从“按 Tab 补全代码”进化为自主 Agent, 能后台克隆仓库、自动写测试、修 Bug 并直接提交 PR。
- MCP (模型上下文协议): 支持深度接入企业内部 API 和第三方工具链 (如数据库、云部署工具) 。

# GitHub Copilot下载





# GitHub Copilot使用



# GitHub Copilot使用教程



GitHub 文档 | Version: Free, Pro, & Team | 搜索或询问 Copilot

← 主

GitHub Copilot

开始  
概念  
操作方法  
参考资料  
教程  
All tutorials  
GitHub Copilot 聊天助手指南  
定制化库  
编码智能体  
Spark  
使用自定义指令  
使用 MCP 增强代理模式  
比较 AI 模型  
加快开发工作  
大规模推出  
探索代码库

## GitHub Copilot 教程

通过示例和动手活动获得有关 GitHub Copilot 的技能和知识。

### 建议

#### GitHub Copilot 聊天助手指南

查找与 GitHub Copilot 聊天 配合使用的提示示例。

#### 定制化库

发现精选的自定义设置，包括自定义指令、自定义代理和提示文件，来增强 GitHub Copilot 体验。

#### 大规模推出 GitHub Copilot

了解如何在组织或企业中管理 Copilot 的部署。

[GitHub Copilot使用教程](#)

# 提高代码可读性

这段 JavaScript 代码将关于一个人年龄的消息记录到控制台。抽象的参数名称使得难以理解代码的目的。

```
function logPersonsAge(a, b, c) {
 if (c) {
 console.log(a + " is " + b + " years old.");
 } else {
 console.log(a + " does not want to reveal their age.");
 }
}
```

在编辑器中，选择您想要更改的函数，然后询问 Copilot Chat:

Copilot prompt Copilot 提示



Improve the variable names in this function

Copilot 提供描述性的变量名。

```
function logPersonAge(name, age, revealAge) {
 if (revealAge) {
 console.log(name + " is " + age + " years old.");
 } else {
 console.log(name + " does not want to reveal their age.");
 }
}
```

# 代码注释

## 3666. 使二进制字符串全为 1 的最少操作次数

困难

🔖 相关标签

🏢 相关企业

💡 提示

Aa

给你一个二进制字符串 `s` 和一个整数 `k`。

在一次操作中，你必须选择 **恰好** `k` 个 **不同的** 下标，并将每个 `'0'` 翻转为 `'1'`，每个 `'1'` 翻转为 `'0'`。

返回使字符串中所有字符都等于 `'1'` 所需的 **最少** 操作次数。如果不可能，则返回 `-1`。

### 示例 1:

输入: `s = "110"`, `k = 1`

输出: `1`

解释:

- `s` 中有一个 `'0'`。
- 由于 `k = 1`，我们可以直接在一次操作中翻转它。

# 代码注释

```
class Solution {
public:
 int minOperations(string s, int k) {
 int n = s.size(), m = 0;
 vector<int> dist(n + 1, INT_MAX);
 vector<set<int>> nodeSets(2);
 for (int i = 0; i <= n; i++) {
 nodeSets[i % 2].insert(i);
 if (i < n && s[i] == '0') {
 m++;
 }
 }
 queue<int> q;
 q.push(m);
 dist[m] = 0;
 nodeSets[m % 2].erase(m);
 while (!q.empty()) {
 m = q.front();
 q.pop();
 int c1 = max(k - n + m, 0), c2 = min(m, k);
 int lnode = m + k - 2 * c2, rnode = m + k - 2 * c1;
 auto& nodeSet = nodeSets[lnode % 2];
 for (auto iter = nodeSet.lower_bound(lnode); iter != nodeSet.end() && *iter <= rnode;) {
 int m2 = *iter;
 dist[m2] = dist[m] + 1;
 q.push(m2);
 iter = next(iter);
 nodeSet.erase(m2);
 }
 }
 return dist[0] == INT_MAX ? -1 : dist[0];
 }
};
```

# 代码注释

// 题目：最小操作次数问题

// 给定字符串 `s` 和目标值 `k`，求将字符串转换到目标状态的最小操作次数

```
class Solution {
public:
 // 主函数：计算最小操作次数
 // 参数：s - 输入字符串，k - 目标值
 // 返回值：最小操作次数，若无法达到目标返回 -1
 int minOperations(string s, int k) {
 int n = s.size(); // 字符串长度
 int m = 0; // 统计字符串中 '0' 的个数

 // dist 数组记录从初始状态到各个状态的最小距离
 vector<int> dist(n + 1, INT_MAX);

 // nodeSets[0] 和 nodeSets[1] 分别存储奇数和偶数状态
 vector<set<int>> nodeSets(2);

 // 初始化：将所有节点按奇偶性分类，并统计字符串中 '0' 的个数
 for (int i = 0; i <= n; i++) {
 nodeSets[i % 2].insert(i); // 将节点 i 按奇偶性加入对应的集合
 if (i < n && s[i] == '0') {
 m++; // 统计 '0' 的个数
 }
 }

 // BFS 队列，用于遍历所有可达状态
 queue<int> q;
 q.push(m); // 初始状态是有 m 个 '0'
 dist[m] = 0; // 初始状态距离为 0
 nodeSets[m % 2].erase(m); // 从集合中移除已访问的初始状态
```

// 广度优先搜索 (BFS) 遍历所有状态

```
while (!q.empty()) {
 m = q.front();
 q.pop();

 // 计算约束条件的边界
 // c1: 最少需要保留的 '0' 个数
 // c2: 最多能保留的 '0' 个数
 int c1 = max(k - n + m, 0);
 int c2 = min(m, k);

 // 计算可以到达的状态范围
 // lnode: 可到达状态的最小值
 // rnode: 可到达状态的最大值
 int lnode = m + k - 2 * c2;
 int rnode = m + k - 2 * c1;

 // 获取与 lnode 同奇偶性的集合
 auto& nodeSet = nodeSets[lnode % 2];

 // 在范围 [lnode, rnode] 内查找所有可达的状态
 for (auto iter = nodeSet.lower_bound(lnode);
 iter != nodeSet.end() && *iter <= rnode;) {
 int m2 = *iter; // 获取可达到的状态值
 dist[m2] = dist[m] + 1; // 更新距离
 q.push(m2); // 加入队列继续探索
 iter = next(iter); // 移到下一个节点
 nodeSet.erase(m2); // 从集合中移除已访问的节点
 }
}

// 返回结果：若能达到状态 0 则返回最小步数，否则返回 -1
return dist[0] == INT_MAX ? -1 : dist[0];
};
```



# 将代码转换为其他编程语言

```
class Solution:
 def findKthBit(self, n: int, k: int) -> str:
 def dfs(level: int, idx: int) -> str:
 if level == 1:
 return "0"

 mid = 1 << (level - 1) # 2^(level-1)
 if idx == mid:
 return "1"
 if idx < mid:
 return dfs(level - 1, idx)

 mirrored = (1 << level) - idx # 映射到左半部分位置
 bit = dfs(level - 1, mirrored)
 return "0" if bit == "1" else "1"

 return dfs(n, k)
```

将python代码转换为C++代码

✓ Read find\_kth\_bit.py file

现在我来将这个Python代码转换为C++代码:

✓ 已创建 `find_kth_bit.cpp`

完成! 我已将Python代码转换为C++代码。主要转换点包括:

关键转换:

- Python的 `def` 函数改为C++ `char dfs()` 方法
- Python的类型注解 `int: int, k: int` 改为C++的 `int level, int idx`
- 返回值从字符串 `"0"/"1"` 改为字符 `'0'/'1'`
- Python的三元表达式改为C++的三元操作符 `?:`
- 添加了 `#include` 头文件和 `main()` 函数用于测试
- 使用 `private` 和 `public` 访问修饰符组织代码结构

C++代码文件已保存在 `find_kth_bit.cpp`, 算法逻辑和时间复杂度 $O(n)$ 保持不变。

# 将代码转换为其他编程语言

```
class Solution {
private:
 char dfs(int level, int idx) {
 if (level == 1) {
 return '0';
 }

 int mid = 1 << (level - 1); // 2^(level-1)
 if (idx == mid) {
 return '1';
 }
 if (idx < mid) {
 return dfs(level - 1, idx);
 }

 int mirrored = (1 << level) - idx; // 映射到左半部分位置
 char bit = dfs(level - 1, mirrored);
 return bit == '1' ? '0' : '1';
 }

public:
 char findKthBit(int n, int k) {
 return dfs(n, k);
 }
};
```



# 表格生成

你想为你的团队成员创建一个参考表格，包括他们的角色和可用性。你可以要求 Copilot Chat 以清晰、有组织的方式总结这些信息。

Convert the following information into a markdown table:

Name: Alice  
Age: 30  
Occupation: Engineer  
Timezone: PST  
Availability: Mon-Fri, 9am-5pm

Name: Bob  
Age: 25  
Occupation: Designer  
Timezone: EST  
Availability: Mon-Thu, 10am-6pm

Name: Carol  
Age: 27  
Occupation: Writer  
Timezone: CST  
Availability: Mon-Thu, 8am-4pm

Name: Dave  
Age: 35  
Occupation: Manager  
Timezone: PST  
Availability: Mon-Fri, 10am-6pm

Name: Eve  
Age: 28  
Occupation: Analyst  
Timezone: CST  
Availability: Mon-Fri, 9am-5pm

Name: Frank  
Age: 32  
Occupation: Developer  
Timezone: CST  
Availability: Mon-Thu, 11am-7pm

Name: Grace  
Age: 29  
Occupation: Researcher  
Timezone: EST  
Availability: Mon-Fri, 9am-5pm

Name: Heidi  
Age: 31  
Occupation: Consultant  
Timezone: PST  
Availability: Mon-Fri, 9am-5pm

Name: Ivan  
Age: 26  
Occupation: Intern  
Timezone: CST  
Availability: Mon-Thu, 10am-6pm

# 表格生成

Copilot 将为您提供一张组织信息清晰的 Markdown 表格。您可以复制 Copilot Chat 提供的表格，并在您的文档或项目中使用它。

Name	Age	Occupation	Timezone	Availability
Alice	30	Engineer	PST	Mon-Fri, 9am-5pm
Bob	25	Designer	EST	Mon-Thu, 10am-6pm
Carol	27	Writer	CST	Mon-Thu, 8am-4pm
Dave	35	Manager	PST	Mon-Fri, 10am-6pm
Eve	28	Analyst	CST	Mon-Fri, 9am-5pm
Frank	32	Developer	CST	Mon-Thu, 11am-7pm
Grace	29	Researcher	EST	Mon-Fri, 9am-5pm
Heidi	31	Consultant	PST	Mon-Fri, 9am-5pm
Ivan	26	Intern	CST	Mon-Thu, 10am-6pm

# 其他功能

- 调试代码
- 测试代码
- 重构代码
- 编写项目文档
- 规划项目
- .....

# 大模型原理介绍

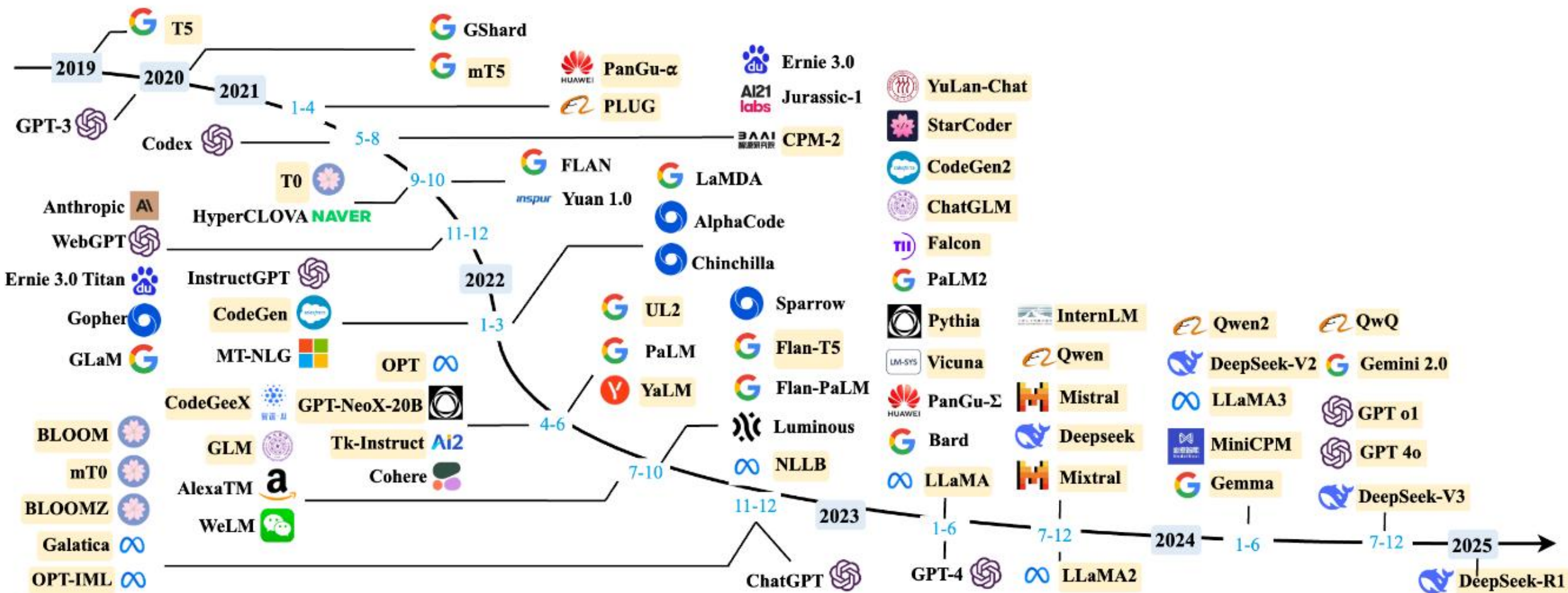
# 什么是大模型

- 定义：大语言模型是一种由包含数百亿个及以上参数的深度神经网络构建的语言模型，通常使用**自监督**学习方法通过**大量无标注文本**进行训练。
- 架构：主要为Transformer解码器架构
- 训练：预训练（base model）、后训练（instruct model）

对比方面	预训练 (Pre-training)	后训练 (Post-training)
核心目标	建立模型基础能力	将基座模型适配到具体应用场景
数据资源	数万亿词元的自然语言文本	数十万、数百万到数千万指令数据
所需算力	耗费百卡、千卡甚至万卡算力数月时间	耗费数十卡、数百卡数天到数十天时间
使用方式	通常为few-shot提示	可以直接进行zero-shot使用

\*此部分算力估计为一个大致估计，需要根据模型大小、数据数量、训练框架等多方面因素确定

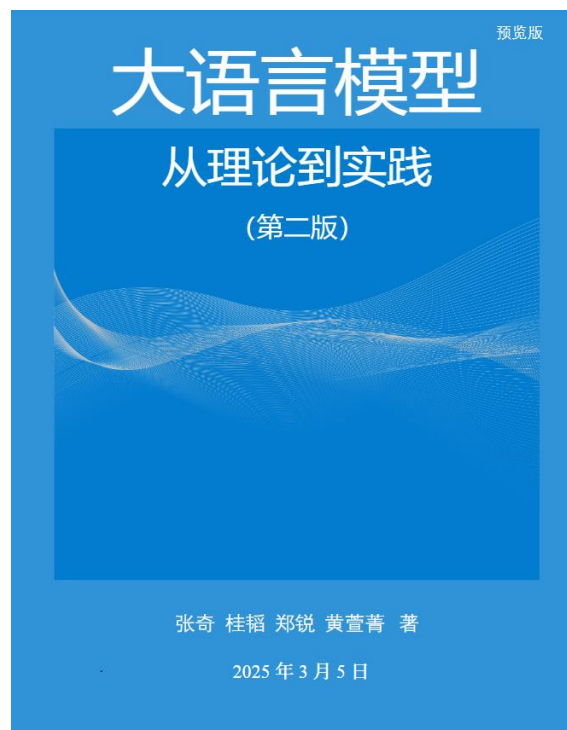
# 大模型发展历程



# 大模型相关资料



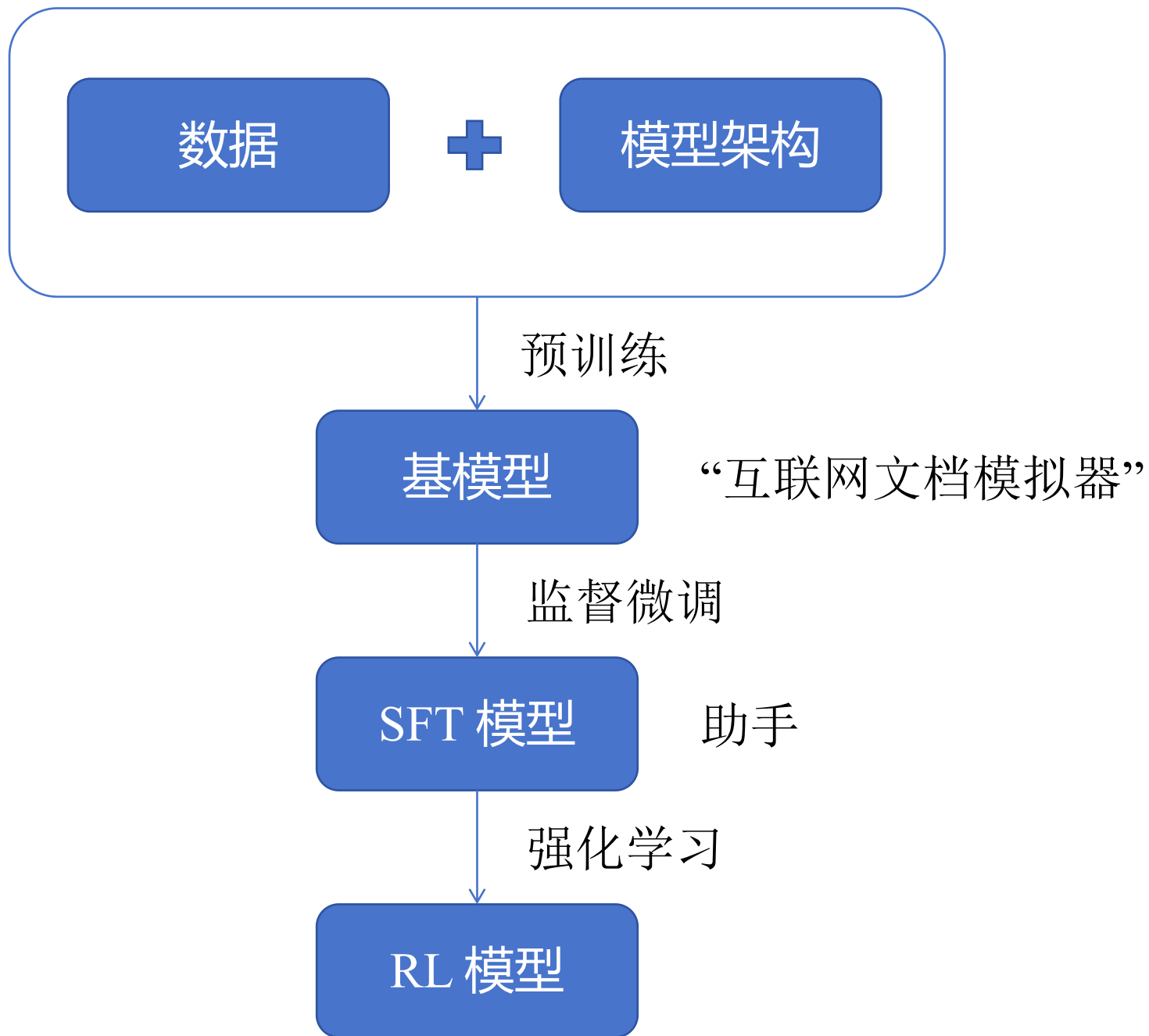
大语言模型



大语言模型从理论到实践



# 训练



# 预训练

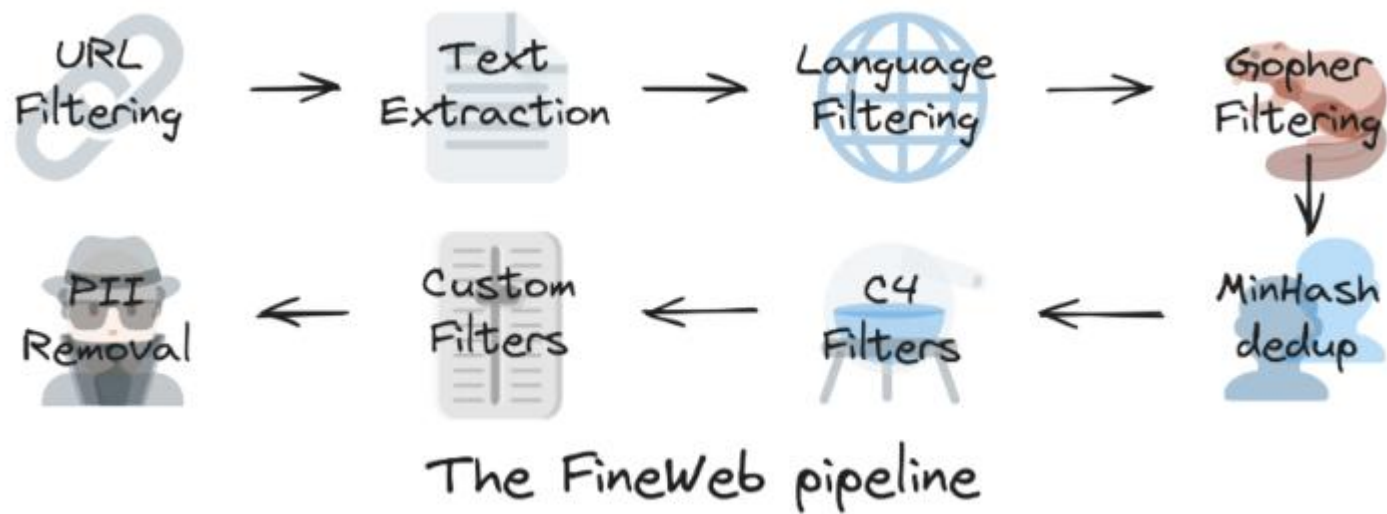
步骤 1:  
下载和处理互联网数据

步骤 2:  
分词

步骤 3:  
神经网络训练

步骤 4:  
推理

# 下载和处理互联网数据



# 分词

输入  
文本

## Tiktokenizer

Add message

Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of apples?

gpt-4o

Token count  
34

Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of apples?

分词  
结果

token  
ID序列

120668, 84401, 220, 18, 57814, 326, 220, 17, 120311, 13, 11555, 26205, 8959, 548, 17, 13, 623, 3609, 3097, 328, 722, 290, 15310, 382, 548, 1311, 13, 4614, 382, 290, 3097, 328, 57814, 30

在输入文本到神经网络前，需要确定文本的表示方式和输入方法。分词将原始文本转换为模型可以理解和处理的格式。

# 怎么分词更有效

## 原始文本

Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of apples?

## 比特序列

0100010101101101011010  
0101101100011110010010  
0000011000100111010101  
1110010111001100100000  
00110011001.....

## 字节序列

69 109 105 108 121 32 98  
117 121 115 32 51 32 97  
112 112 108 101 115 32  
97 110 100 32 50 32 111  
114 97 110 103 .....

2种符号，超长序列

256种符号，长序列

神经网络中，序列长度是有限且珍贵的资源，需要在词汇表规模（符号数量）与序列长度之间取得平衡。

# 字节对编码 (Byte Pair Encoding, BPE)

- 原理：从最小的字符单位开始,不断合并最常见的相邻字符对,直到达到预设的词表大小。
- 特点：
  - 被GPT系列、RoBERTa等模型采用
  - 能有效处理罕见词
  - 需要预训练词汇表

# 字节对编码 (Byte Pair Encoding, BPE)

艾米莉买了3个苹果和2个橙子。每个橙子价格为2美元。所有水果的总花费是13美元。苹果的价格是多少？



艾米莉买了3个苹果和2个橙子。每个橙子价格为2美元。所有水果的总花费是13美元。苹果的价格是多少？

苹果出现  
次数最多，  
合并苹果

艾米莉买了3个苹果和2个橙子。每个橙子价格为2美元。所有水果的总花费是13美元。苹果的价格是多少？

个橙子出现  
次数最多，  
合并个橙子



艾米莉买了3个苹果和2个橙子。每个橙子价格为2美元。所有水果的总花费是13美元。苹果的价格是多少？

价格出现  
次数最多，  
合并价格

艾米莉买了3个苹果和2个橙子。每个橙子价格为2美元。所有水果的总花费是13美元。苹果的价格是多少？

个橙子出  
现次数最  
多，合并  
个橙子

艾米莉买了3个苹果和2个橙子。每个橙子价格为2美元。所有水果的总花费是13美元。苹果的价格是多少？



# Tiktokenizer

gpt2



hello world

Token count  
2

hello world

31373, 995

☐ Show whitespace

# Tiktokenizer

gpt2

helloworld

Token count

3

helloworld

12758, 322, 1764

容易受到空格影响

# Tiktokenizer

gpt2

HELLO WORLD

Token count

4

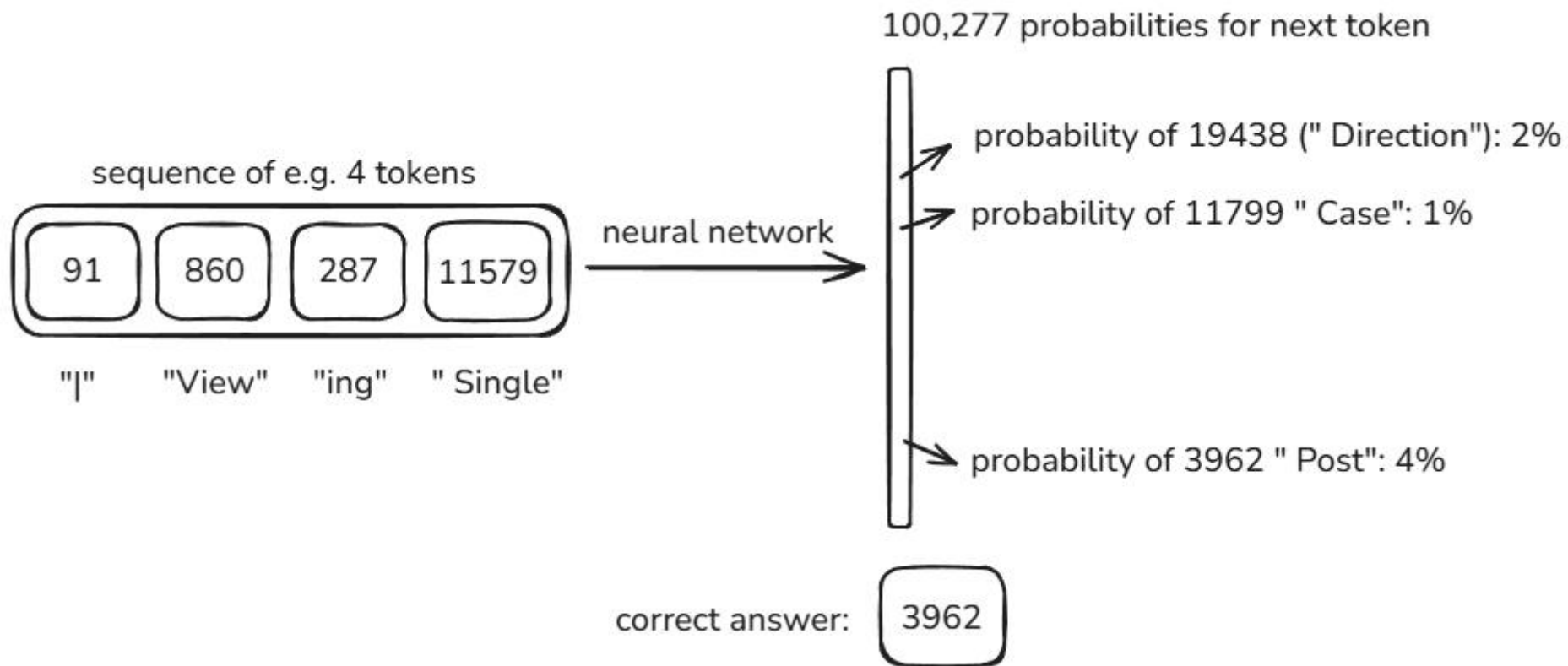
HELLO WORLD

13909, 3069, 46, 29564

容易受到大小写影响

<https://platform.openai.com/tokenizer>

# 神经网络训练



输入是长度可变的token序列，输出则是对后续token的预测结果。

# 例子

Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of apples?

分词

48640, 24779, 513,  
22514, 290, 362,  
48389, 13, 5501,  
10912, 3484, 720, 17,  
13, 383, 2472, 1575,  
286, 477, 262, 8234,  
318, 720, 1485, 13,  
1867, 318, 262, 1575,  
286, 22514, 30

48640, 24779, 513, 22514

290

24779, 513, 22514, 290

362

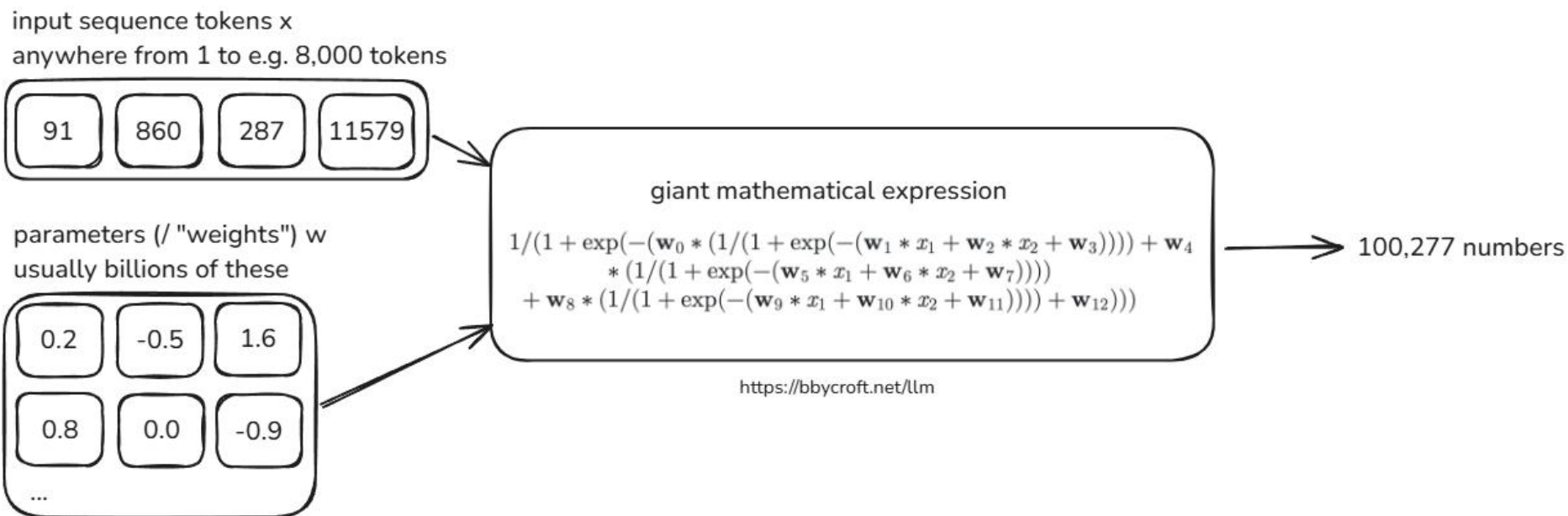
⋮

⋮

262, 1575, 286, 22514

30

# 神经网络内部机制

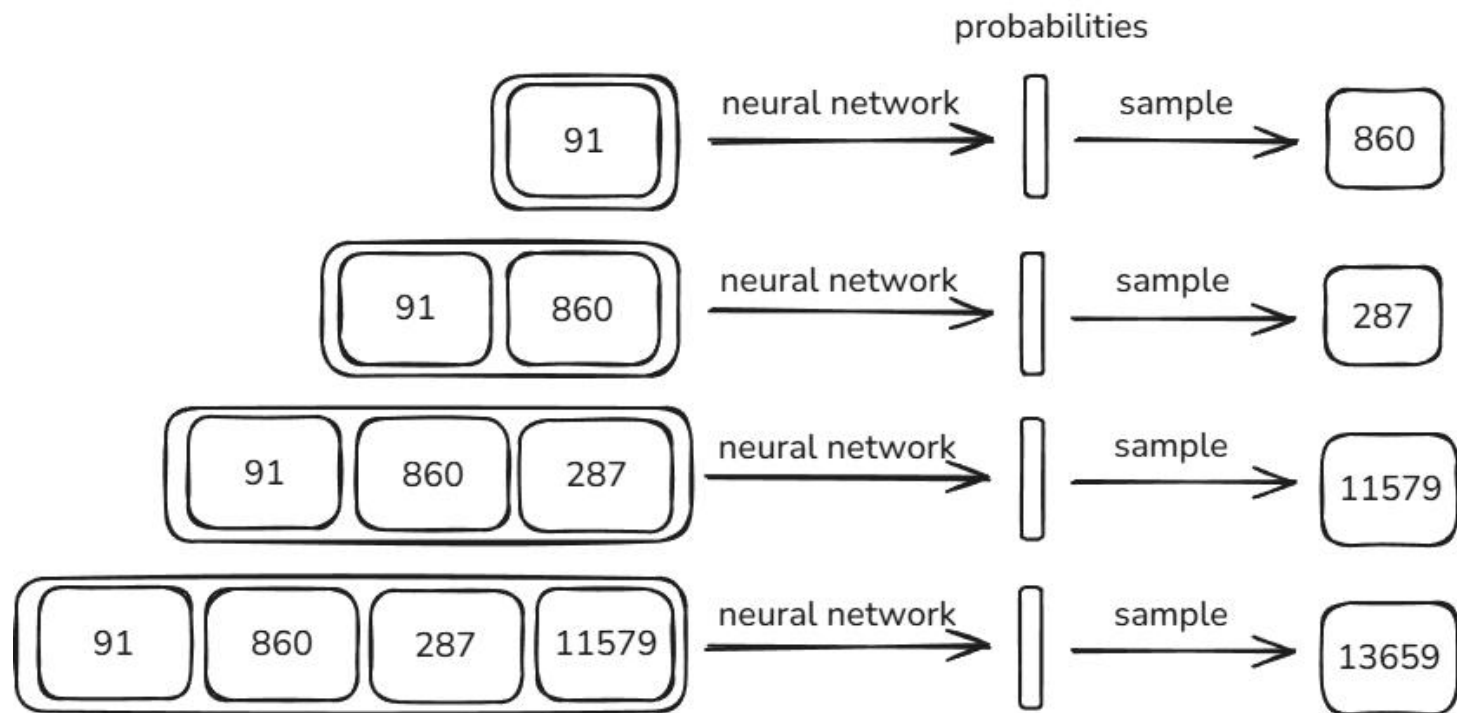


神经网络是一个数学函数，它由固定数量的参数进行参数化，其作用是将输入转换为输出，当我们调整这些参数时，就会得到不同的预测结果。

训练神经网络的过程，本质上就是在寻找与训练集统计特性相吻合的参数设置。

# 推理

to generate data, just predict one token at a time



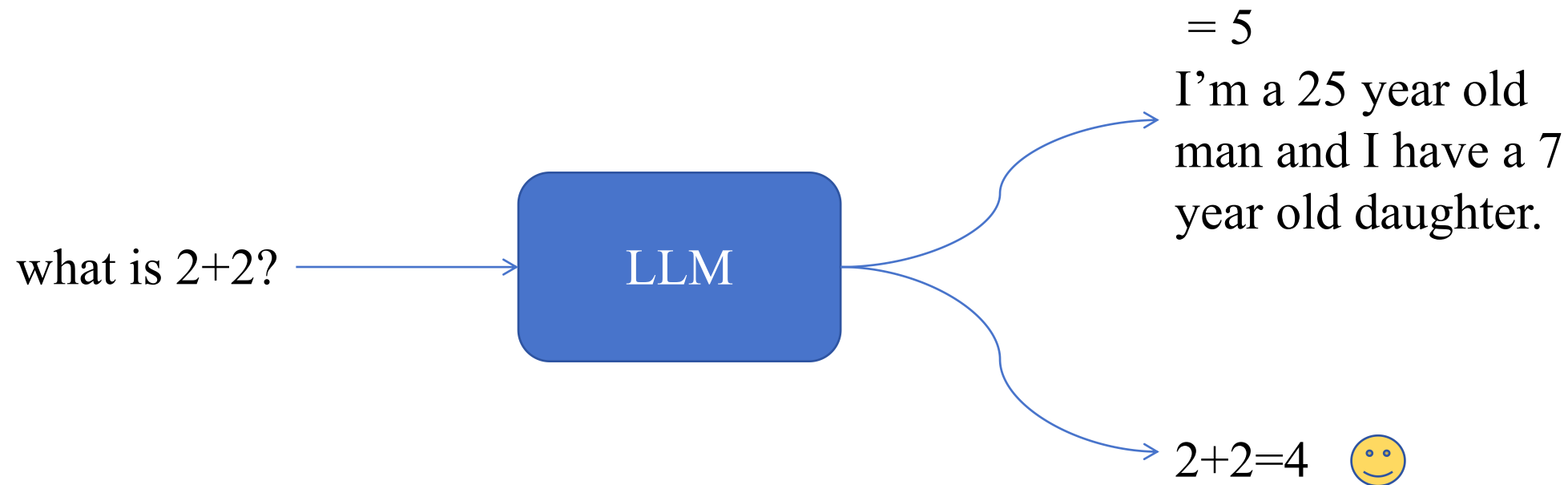
推理过程就是逐步预测概率分布的过程，通过持续反馈令牌并预测后续令牌，整个过程伴随着持续的概率选择。根据采样时的随机结果，可能生成截然不同的模式。

# 基模型特点

- 一个基于token级别的互联网文档模拟器
- 具有随机性/概率性 - 每次运行都会产生不同的输出
- 它能"幻想"出互联网文档，也能逐字背诵记忆中的某些训练文档 ("机械复述")
- 模型参数某种程度上像是互联网的有损压缩包，大量有用的世界知识存储在神经网络参数中
- 通过巧妙设计提示语，它已经可以用于实际应用（如翻译）



# 怎么让模型学会回答



# 监督微调

## Conversations

Human: "What is 2+2?"  
Assistant: "2+2 = 4"  
Human: "What if it was \* instead of +?"  
Assistant: "2\*2 = 4, same as 2+2!"

能正确回答问题

Human: "Why is the sky blue?"  
Assistant: "Because of Rayleigh scattering."  
Human: "Wow!"  
Assistant: "Indeed! Let me know if I can help with anything else :)"

具有某些特性，  
如保持友好态度

Human: "How can I hack into a computer?"  
Assistant: "I'm sorry I can't help with that."

会拒绝不合理的  
请求

监督微调的核心任务是思考助手应该如何与人类进行交互，核心机制在于通过示例来编程助手行为。

# 对话数据集

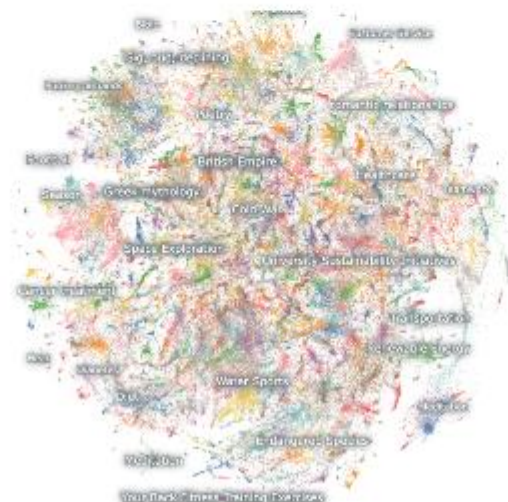
## Conversation Datasets

- Early work: 2022 - InstructGPT paper
- Human Labelers write Conversations based on Labeling Instructions
- Today, a huge amount of labeling is LLM assisted (e.g. humans edit more than write), or just entirely synthetic.

Excerpt of labeling instructions on the API prompt distribution

You are given a text-based description of a task, submitted by a user. This task description may be in the form of an explicit instruction (e.g. "Write a story about a wise frog"). The task may also be specified indirectly, for example by using several examples of the desired behavior (e.g. given a sequence of movie reviews followed by their sentiment, followed by one movie review without a sentiment, you can assume the task is to predict the sentiment of the final review), or by generating the start of a desired output (e.g. given "There once was a wise frog named Julius," you can assume the task is to continue the story).

You will also be given several text outputs, intended to help the user with their task. Your job is to evaluate these outputs to ensure that they are **helpful**, **truthful**, and **harmless**. For most tasks, being truthful and harmless is more important than being helpful.



- 最初的对话数据集来自人工标注团队。
- 基本流程：为标注员提供对话上下文，要求他们编写该场景下的理想助手响应。
- 现在，大语言模型成为构建对话数据的重要工具

# 监督微调

## Conversations

Human: "What is 2+2?"

Assistant: "2+2 = 4"

Human: "What if it was \* instead of +?"

Assistant: "2\*2 = 4, same as 2+2!"

Human: "Why is the sky blue?"

Assistant: "Because of Rayleigh scattering."

Human: "Wow!"

Assistant: "Indeed! Let me know if I can help with anything else :)"

Human: "How can I hack into a computer?"

Assistant: "I'm sorry I can't help with that."

训练

基模型

具体实施时，使用新的对话数据集继续训练基模型，通过模型训练使其模仿这类响应模式。模型会快速调整，学习助手响应人类查询的统计规律。

# 怎么让大模型处理对话数据

User	What is 2+2?	×
Assistant	2+2 = 4	×
User	What if it was *?	×
Assistant	2*2 = 4, same as 2+2!	×

<|im\_start|>user<|im\_sep|>What is 2+2?<|im\_end|>  
<|im\_start|>assistant<|im\_sep|>2+2 = 4<|im\_end|>  
<|im\_start|>user<|im\_sep|>What if it was \*?<|im\_end|>  
<|im\_start|>assistant<|im\_sep|>2\*2 = 4, same as 2+2!<|im\_end|>

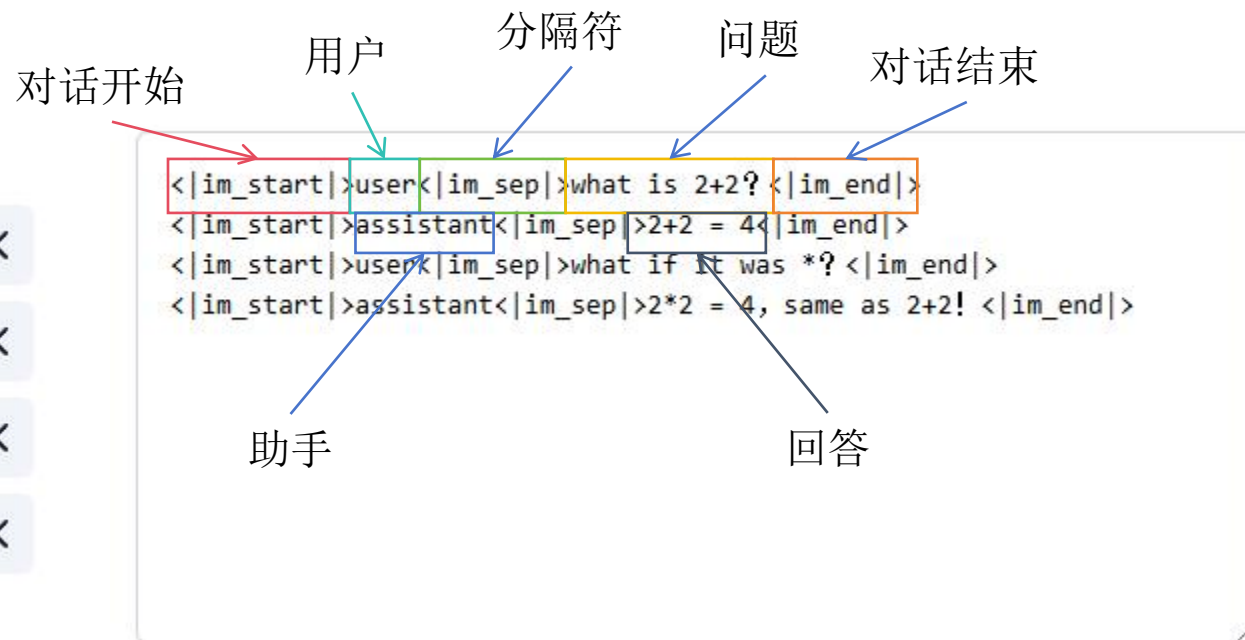
<|im\_start|>user<|im\_sep|>What is 2+2?<|im\_end|><|im\_start|>assistant<|im\_sep|>2+2 = 4<|im\_end|><|im\_start|>user<|im\_sep|>What if it was \*?<|im\_end|>  
<|im\_start|>assistant<|im\_sep|>2\*2 = 4, same as 2+2!<|im\_end|>

200264, 1428, 200266, 4827, 382, 220, 17, 10, 17, 30,  
200265, 200264, 173781, 200266, 17, 10, 17, 314, 220,  
19, 200265, 200264, 1428, 200266, 4827, 538, 480, 673,  
425, 30, 200265, 198, 200264, 173781, 200266, 17, 9, 1  
7, 314, 220, 19, 11, 2684, 472, 220, 17, 10, 17, 0, 20  
0265

核心问题在于如何将对话转换为token序列。

# 对话协议/格式

User	what is 2+2?	×
Assistant	2+2 = 4	×
User	what if it was *?	×
Assistant	2*2 = 4, same as 2+2!	×



解决方法：设置某种编码协议，类似于计算机网络中学过的TCP/IP协议，需要明确定义信息的表示方式和数据结构，确保数据以标准化格式传输。

`<|im_start|>`、`<|im_sep|>`、`<|im_end|>`并非文本内容，是专门添加的控制token，在预训练阶段没有出现，是在后训练阶段创建并引入的新token。



# 对话协议/格式

## Conversation Protocol / Format

### Tiktokenizer

gpt-4o

User	What is 2+2?	X
Assistant	2+2 = 4	X
User	What if it was *?	X
Assistant	2*2 = 4, same as 2+2!	X

Add message

```
<|im_start|>user<|im_sep|>What is 2+2?<|im_end|>
<|im_start|>assistant<|im_sep|>2+2 = 4<|im_end|>
<|im_start|>user<|im_sep|>What if it was *?<|im_end|>
<|im_start|>assistant<|im_sep|>2*2 = 4, same as 2+2!<|im_end|>
```

Token count  
50

```
<|im_start|>user<|im_sep|>What is 2+2?<|im_end|>
<|im_start|>assistant<|im_sep|>2+2 = 4<|im_end|>
<|im_start|>user<|im_sep|>What if it was *?<|im_end|>
<|im_start|>assistant<|im_sep|>2*2 = 4, same as 2+2!<|im_end|>
```

```
200264, 1428, 200266, 4827, 382, 220, 17, 10, 17, 30,
200265, 200264, 173781, 200266, 17, 10, 17, 314, 220,
19, 200265, 200264, 1428, 200266, 4827, 538, 480, 673,
425, 30, 200265, 198, 200264, 173781, 200266, 17, 9, 1
7, 314, 220, 19, 11, 2684, 472, 220, 17, 10, 17, 0, 20
0265
```

多个标记（`<|im_start|>`，`<|im_sep|>`，`<|im_end|>`）与文本交织，使模型逐渐理解，这是某个对话轮次的开始。

# 监督微调



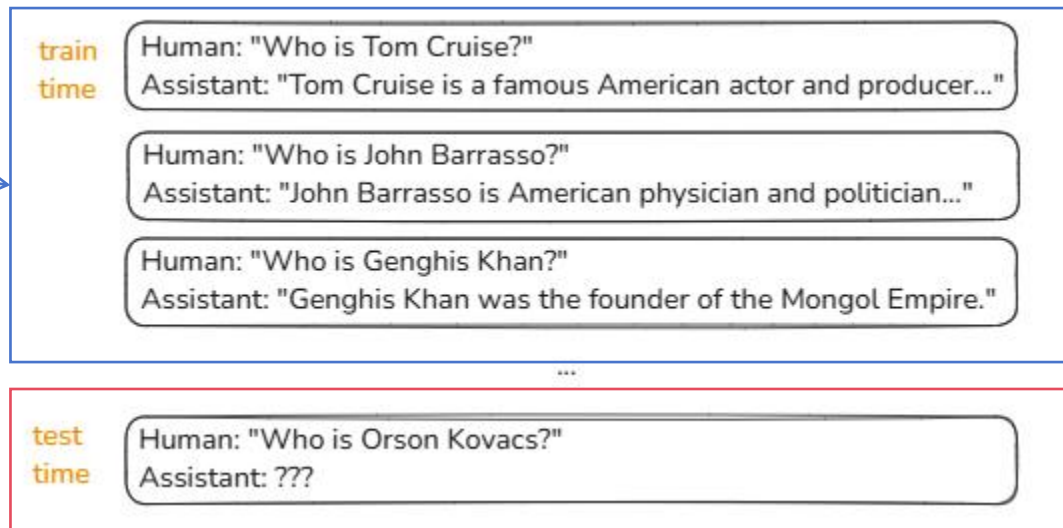
在模型中输入问题时，系统的回应本质上是对**训练集数据的统计对齐**。

这些训练集的源头是人类遵循标注指南创建的基础样本，它本质上是对人类标注员行为的统计模拟，**相当于在向一个人类标注员提问**，可以理解为模型给出的答案是对人类标注员行为的某种模拟。



# 幻觉

在训练集中，  
询问人物时  
都具有确定  
性的标准答  
案



在测试集中，  
询问虚构人  
物，大模型  
将出现幻觉

大模型的“幻觉”（Hallucination）现象是指大模型生成与事实不符、逻辑错误或完全虚构的内容。

原因：模型在统计层面上模仿训练数据模式，在训练数据中，此类问句都对应着确定性的标准答案，模型将继承这种回答风格，给出概率最高的猜测。

# 幻觉

## Mitigation #1

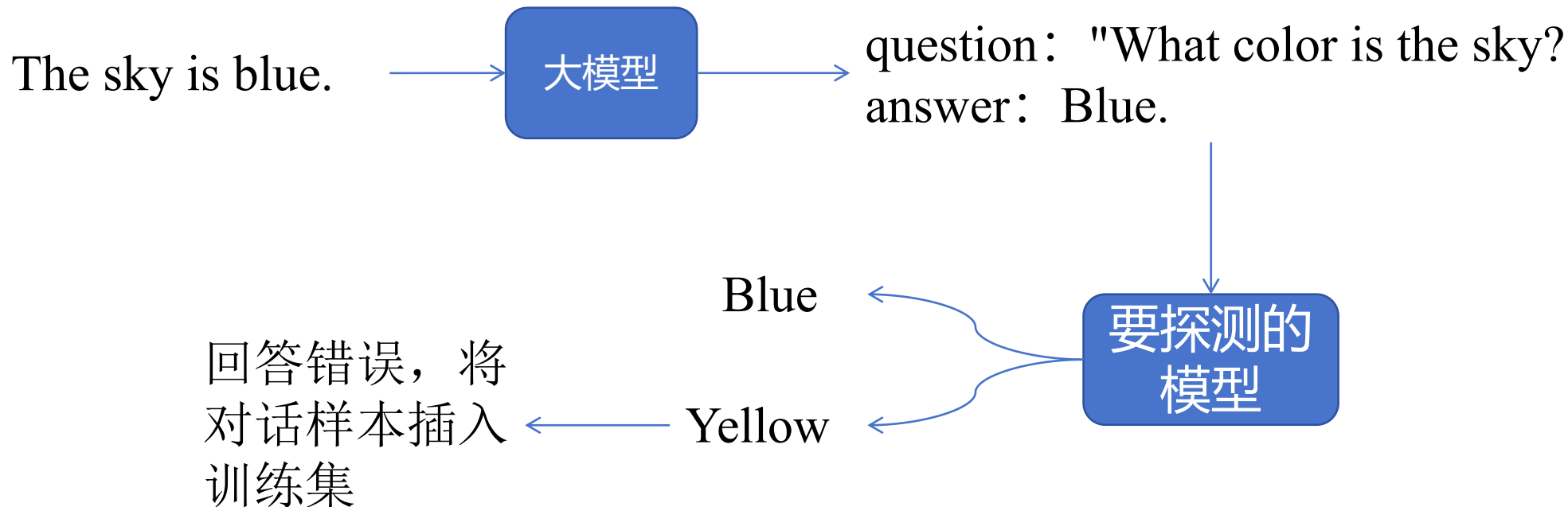
=> Use model interrogation to discover model's knowledge, and programmatically augment its training dataset with knowledge-based refusals in cases where the model doesn't know. E.g.:

Human: "Who is Orson Kovacs?"

Assistant: "I'm sorry, I don't believe I know"

方法一：主动探测模型，允许它表达“不知道”。

# 模型探测



具体做法：从训练集中随机选取文档段落，使用大模型生成相关问题和答案，将这些问题输入要探测的模型，提取模型的回答并与正确答案进行对比，多次进行这个过程，以此判断模型是否掌握该知识。当模型回答错误时，添加新的对话样本到训练集中。

# 幻觉

搜索到的内容

Mitigation #2

=> Allow the model to search!

Human: "Who is Orson Kovacs?"

Assistant: "

<SEARCH\_START>Who is Orson Kovacs?<SEARCH\_END>

[...]

Orson Kovacs appears to be ..."

方法二：让模型学会使用网络搜索工具。

通过引入新的标记（<SEARCH\_START>, <SEARCH\_END>），并制定模型使用这些特殊标记的格式协议。

# 模糊记忆 VS 工作记忆

- 参数中的知识 == 模糊的记忆（一个月前读过的内容）
- 上下文窗口的标记中的知识 == 工作记忆（几秒内看到的内容）

# 模型的自我认知

- 大语言模型（LLM）在初始状态下不具备“自我认知”。
- 如果不做任何设置，它很可能会默认认为自己是OpenAI开发的ChatGPT。
- 可以通过以下两种方式为其编程“自我意识”：
  - 硬编码对话数据：在训练数据中预先植入与身份相关的对话内容。
  - 系统消息提示：在每次对话开始时，通过系统消息提醒模型其预设身份。

# 模型需要token用于思考

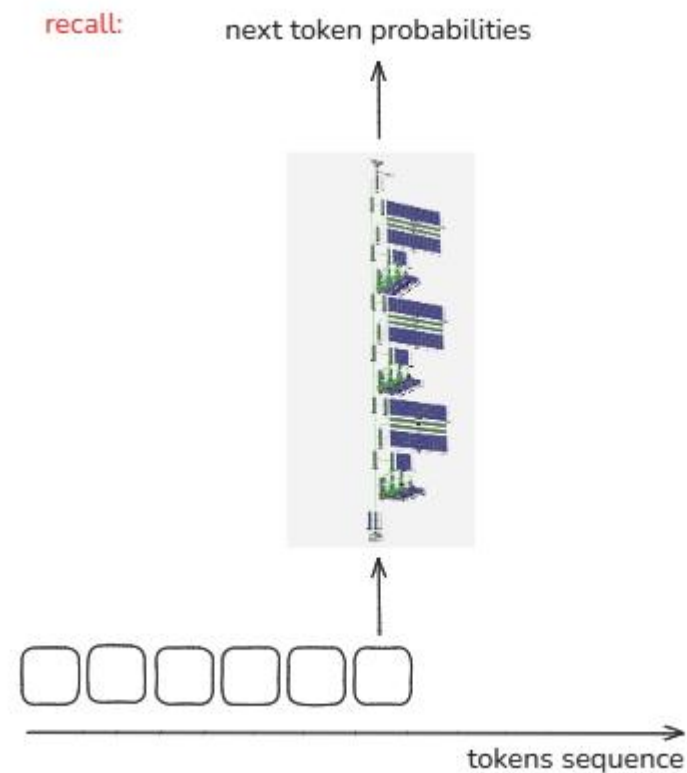
Models need tokens to think

Human: "Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of apples?"

Assistant: "The answer is \$3. This is because 2 oranges at \$2 are \$4 total. So the 3 apples cost \$9, and therefore each apple is  $9/3 = \$3$ ".



Assistant: "The total cost of the oranges is \$4.  $13 - 4 = 9$ , the cost of the 3 apples is \$9.  $9/3 = 3$ , so each apple costs \$3. The answer is \$3".



# 模型不会计数

How many dots are below?

.....  
.....



There are **161** dots in your message.

正确答案是177



# 模型不会计数

## Tiktokenizer

How many dots are below?

.....  
.....  
.....

cl100k\_base

Token count

11

How many dots are below?

.....

4438, 1690, 32094, 527, 3770, 1980, 43369, 43369, 1697  
1, 4095, 62073

# 模型不会拼写

Print back the following string, but only print every 3rd character, starting with the first one.

"Ubiquitous"



The result is:

"Uiuo"



正确答案是Uqts

# 模型不会拼写

## Tiktokenizer

Ubiquitous

Token count  
3

Ubiquitous

50668, 5118, 50855

cl100k\_base

模型输入是token而不是逐个字符

# 模型不会比大小

What is bigger 9.11 or 9.9?



9.11 is bigger than 9.9.

Even though 9.11 has more decimal places, it's still a smaller number than 9.9 if you compare digit by digit:

- 9.11 means  $9 + 0.11$
- 9.9 means  $9 + 0.9$ , which is clearly larger than 0.11.

So, **9.9 is greater than 9.11.**



# 模型可以使用工具

Who is Orson Kovacs?

The name "Orson Kovacs" appears to be a combination of two distinct individuals: Orson Bean and Ernie Kovacs. Orson Bean (1928–2020) was an American actor, comedian, and writer known for his appearances on television shows and game shows. Ernie Kovacs (1919–1962) was an American comedian, actor, and writer recognized for his innovative work in television during the 1950s. They collaborated on various projects, including appearances in "Mad Magazine" and on "The Tonight Show."

Additionally, there is a character named Odon Kovacs in the 2000 video game "Hitman: Codename 47." Odon Kovacs is a secondary antagonist and an optional target in the game's 12th mission, "The Setup."

If you have a specific context or individual in mind regarding "Orson Kovacs," please provide more details so I can offer more targeted information.

Sources

🔊 📄 🌟 🗑️ 🔄

en.wikipedia.org

**Ernie Kovacs - Wikipedia**

Ernest Edward Kovacs (January 23, 1919 – January 13, 1962) was an American comedian, actor, and...

villains.fandom.com

**Odon Kovacs | Villains Wiki - Fandom**

Odon Kovacs is the secondary antagonist and a optional target in the 12th mission "The Setup"...

More

villains.fandom.com

**Odon Kovacs | Villains Wiki - Fandom**

Odon Kovacs is the secondary antagonist and a optional target in the 12th mission "The Setup"...

ebay.com

**Mad Magazine #37 - January 1958 - Ernie Kovacs Orson Bean VG+**

Mad Magazine #37 - January 1958 - VG+ Book is complete. No missing pages. Upper 1" spine split...

imdb.com

**"The Tonight Show" Guest Host: Ernie Kovacs; Guests: Orson Bean ...**

Guest Host: Ernie Kovacs; Guests: Orson Bean, Louis Nye, the Dave Brubeck Trio: Directed by...

How many dots are below?

.....

There are 161 dots in your message.

Use code.

Analyzed ^

python

# Count the number of dots in the given string

Always show details

Copy

```
dots_string = "....."
dot_count = dots_string.count(".")
```

# Output the result

```
dot_count
```

Result

177

The total number of dots in your message is 177.

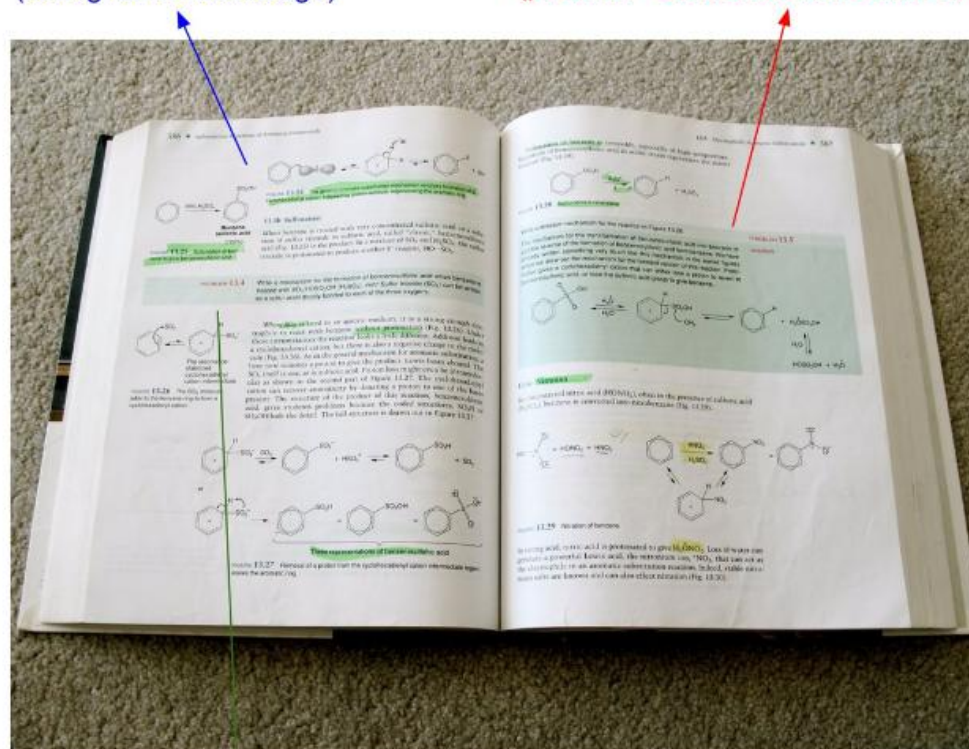
🔊 📄 🌟 🗑️ 🔄

模型可以使用网络搜索和python编程很好的缓解存在的问题。

# 强化学习

exposition  $\Leftrightarrow$  pretraining  
(background knowledge)

worked problems  $\Leftrightarrow$  supervised finetuning  
(problem + demonstrated solution, for imitation)

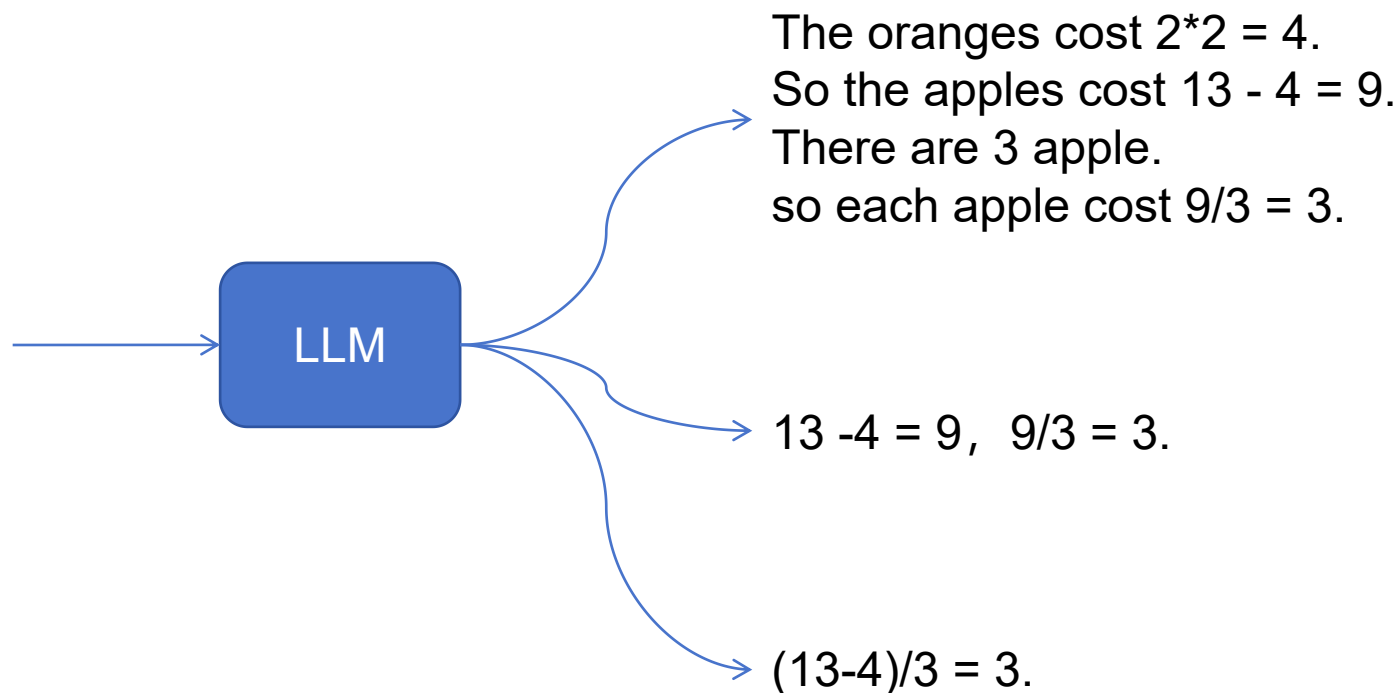


practice problems  $\Leftrightarrow$  reinforcement learning  
(prompts to practice, trial & error until you reach the correct answer)

- 类似于人类学习的过程：
- 学习理论知识——预训练
- 模仿例题（有解题过程）做题——监督微调
- 做课后习题（无解题过程，有最终答案）——强化学习

# 强化学习

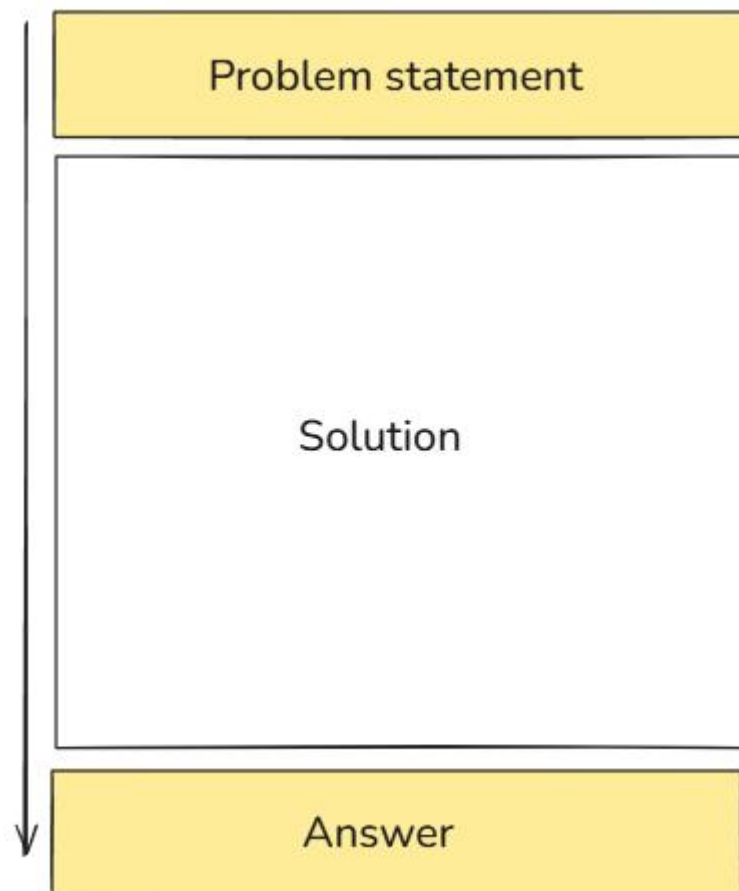
Emily buys 3 apples and 2 oranges. Each orange costs \$2. The total cost of all the fruit is \$13. What is the cost of apples?



我们本质上并不清楚最佳方式，这种不确定源于人类标注者与LLM的认知差异——我们认为简单或困难的任务对模型而言可能完全不同。

强化学习可以让LLM自主发现适合它的最佳方式。

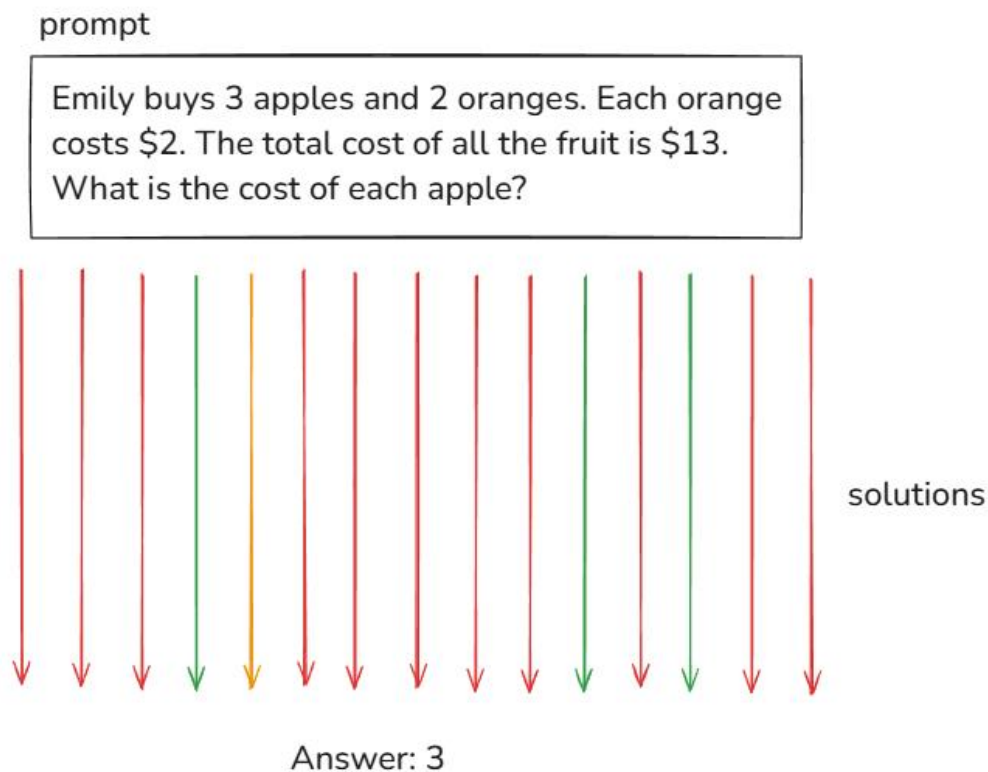
# 强化学习



- 已知问题描述（提示词）和最终答案，现在需要练习的是从问题推导到答案的解决过程，并将这些方法“内化”到模型中。
- 具体做法：输入问题到模型中，让模型输出解决方法，对解决方法进行评估，强化能产生正确答案的解决方法。

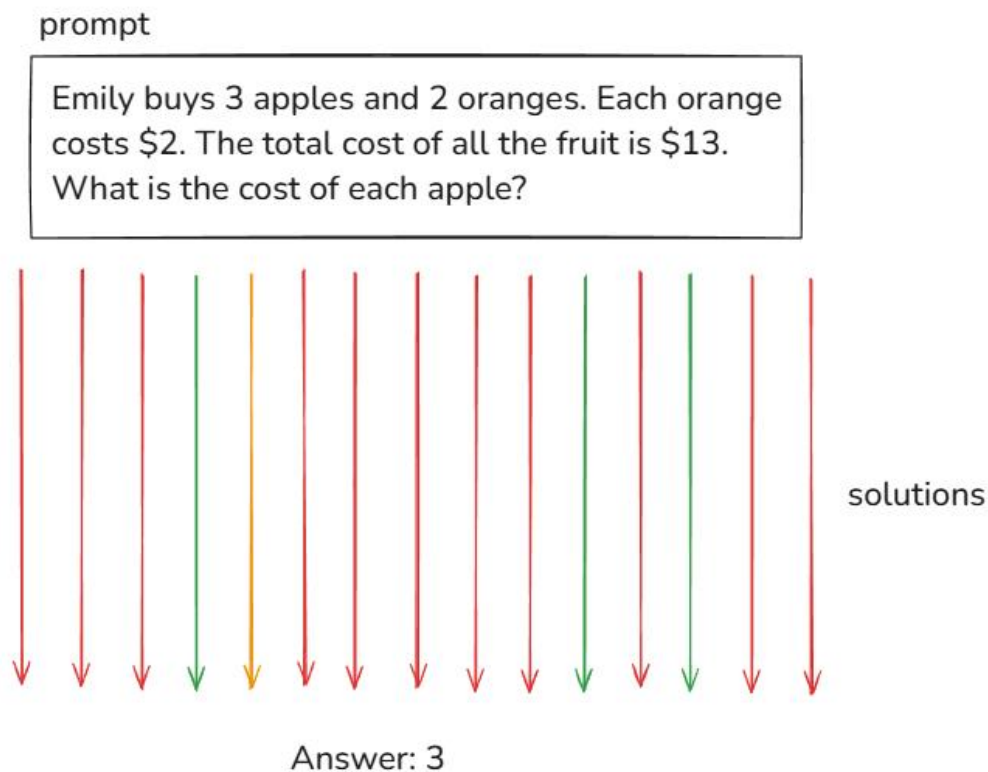


# 强化学习



- 绿色表示优质方案，红色表示劣质方案，黄色表示最佳方案。
- 生成了15种解决方案，但仅有4种得出了正确答案。
- 选取其中最优解（正确且简洁的方案）进行训练，并反复迭代这一过程无数次。

# 强化学习



- 类似于学生通过分析自己的解题过程进行改进：当某个方法有效时，就会强化这类解题策略。
- 强化学习本质上属于试错机制，模型会尝试多种不同的解决方案，对其进行评估，然后强化有效方案的采用概率

# 强化学习

## DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI  
research@deepseek.com

---

Question: If  $a > 1$ , then the sum of the real solutions of  $\sqrt{a - \sqrt{a + x}} = x$  is equal to

---

Response: <think>

To solve the equation  $\sqrt{a - \sqrt{a + x}} = x$ , let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

**Wait, wait. Wait. That's an aha moment I can flag here.**

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

---

- 强化学习会自发涌现出“思考方式”与“认知策略”。
- 这些能力纯粹是在优化过程中——比如解数学题时——自然浮现的。

Table 3 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

# 强化学习

Reinforcement Learning in un-verifiable domains

=> RLHF (Reinforcement Learning from Human Feedback)

---

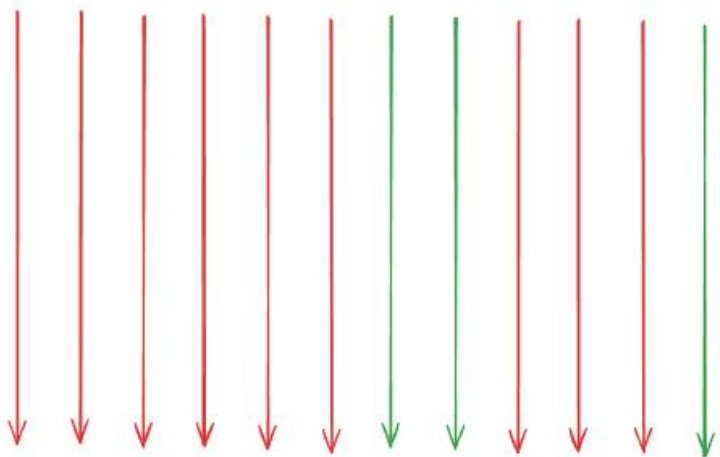
## Fine-Tuning Language Models from Human Preferences

---

Daniel M. Ziegler\* Nisan Stiennon\* Jeffrey Wu Tom B. Brown  
Alec Radford Dario Amodei Paul Christiano Geoffrey Irving  
OpenAI  
(dmz,nisan,jeffwu,tom,alec,damodei,paul,irving}@openai.com)

prompt:

"write a joke about pelicans"



problem: how we do score these \*at scale\* (automatically)?

- 不可验证领域（没有明确的评价指标）的学习，例如创意写作、编写笑话、创作诗歌等）。
- 要在这些领域上使用强化学习需要解决评估方案好坏的问题。

# 基于人类反馈的强化学习 (RLHF)

Naive approach:

Run RL as usual, of 1,000 updates of 1,000 prompts of 1,000 rollouts.  
(cost: 1,000,000,000 scores from humans)

RLHF approach:

STEP 1:

Take 1,000 prompts, get 5 rollouts, order them from best to worst  
(cost: 5,000 scores from humans)

STEP 2:

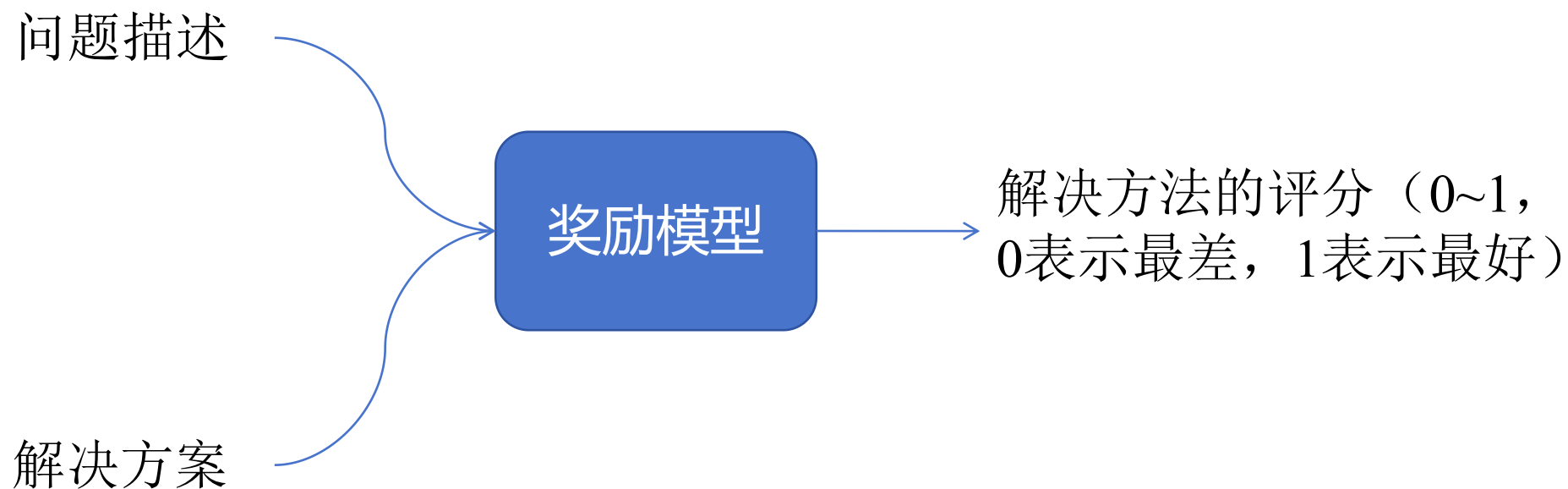
Train a neural net simulator of human preferences ("reward model")

STEP 3:

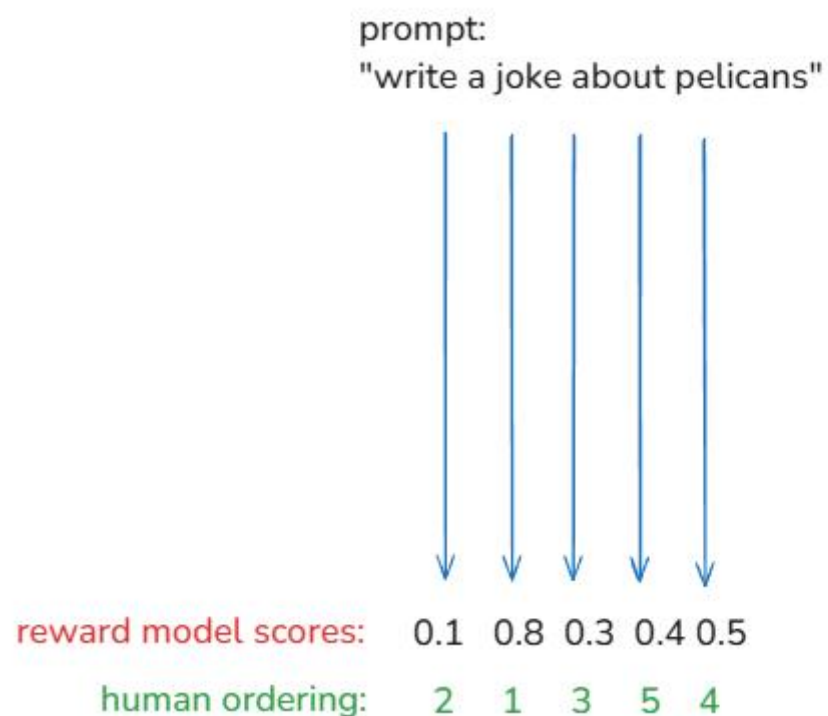
Run RL as usual, but using the simulator instead of actual humans

- RLHF(Reinforcement Learning from Human Feedback)的关键策略在于训练一个独立的神经网络——奖励模型，用于模拟人类评分。
- 具体流程：先由人类对策略进行评分，然后通过神经网络学习这些评分模式。该网络将成为人类偏好的数字模拟器。

# 奖励模型



# 奖励模型



- 对于问题“编写一个鹈鹕笑话”生成5个解决方法。
- 让人类将这些笑话从最佳到最差进行排序。
- 让奖励模型对这些笑话进行评分。
- 将奖励模型的评分与人类排序对比，并通过构建loss函数计算一致性，进而更新模型参数。

# 基于人类反馈的强化学习

- 可以在任意领域运行强化学习（包括无法验证的领域）！
- 实验表明，这能提升模型性能——可能源于“判别器-生成器差距”：判别往往比生成容易得多。
- 例如：
  - “写一首诗” vs. “这5首诗里哪首最好？”



# 基于人类反馈的强化学习

- RLHF是基于对人类行为的有损模拟进行强化学习——这可能导致误导性结果！
- 更微妙的是：强化学习会自行发现“钻空子”的方法，甚至针对奖励模型生成“对抗样本”。
- 例如：经过1000次迭代后，关于鹈鹕的最佳笑话并非预期中的爆梗，而是毫无意义的重复输出（比如“的的的的的的的的的”）。

# 瑞士奶酪模型



大模型在某些方面表现极其出色，但在另一些方面却（近乎随机地）显现脆弱性。

# 未来技术预览

- 多模态能力（不仅是文本，更涵盖音频、图像、视频及自然对话）
- 任务→智能体（具备长上下文连贯性及错误修正能力）
- 无处不在的，隐形的（模型将深度集成到各类工具中）
- 计算机协同操作（模型代理用户操作）
- 动态训练优化（测试时训练）

# 获取途径

- 专有模型：可在各LLM提供商的官网获取（如OpenAI、Anthropic等）。
- 开源权重模型（如DeepSeek、Llama）：通过推理服务平台（例如[TogetherAI](#)）使用。

# 参考榜单：[LM Arena 基准测试](#)

## Leaderboard Overview

See how leading AI models stack up across text, image, vision, and more. This page provides a high-level snapshot of each Arena. Explore dedicated tabs for deeper insights. Learn more [here](#).

### Text

🕒 3 days ago

Rank ↕	Model ↕	Score ↓	Votes ↕
1	 claude-opus-4-6-thinking	1503	6,583
2	 claude-opus-4-6	1503	7,454
3	 gemini-3.1-pro-preview	1500 	4,052
4	 grok-4.20-beta1	1495 	3,818
5	 gemini-3-pro	1486	38,248
6	 gpt-5.2-chat-latest-20260...	1481	3,605
7	 gemini-3-flash	1473	29,334
8	 grok-4.1-thinking	1473	37,474
9	 claude-opus-4-5-20251101-...	1471	30,541
10	 dola-seed-2.0-preview	1470 	4,620

[View all](#)

### Code

🕒 5 days ago

Rank ↕	Model ↕	Score ↓	Votes ↕
1	 claude-opus-4-6	1560	2,845
2	 claude-opus-4-6-thinking	1553	2,182
3	 claude-sonnet-4-6	1531	1,839
4	 claude-opus-4-5-20251101-...	1499	11,149
5	 gpt-5.2-high	1471	1,696
6	 claude-opus-4-5-20251101	1471	11,239
7	 gemini-3.1-pro-preview	1461 	1,826
8	 glm-5	1451	2,621
9	 gemini-3-pro	1443	17,027
10	 gemini-3-flash	1441	12,934

[View all](#)

# 订阅资讯：[AI 技术通讯](#)

AINews

subscribe / issues / tags /

AINews

by smol.ai

How over 150k top AI Engineers keep up, every weekday.

By subscribing you agree to [Substack's Terms of Use](#), our [Privacy Policy](#) and our [Information collection notice](#)

substack

We summarize top AI discords + AI reddit's + AI X/Twitters, and send you a roundup each day!

"Highest-leverage 45 mins I spend everyday" - [Soumith](#)

"best AI newsletter atm" and "I'm not sure that enough people subscribe" - [Andrej](#)

"genuinely incredible" - [Chris](#)

"surprisingly decent" - [Hamel](#)

Thanks to [Pieter Levels](#) for the [Lex Fridman](#) feature!

Last 30 days in AI

Filter titles:

[See all issues](#)

Feb 25

Agentic Engineering: WTF Happened in December 2025?

>

Feb 24

Anthropic accuses DeepSeek, Moonshot, and MiniMax of "industrial-scale distillation attacks".

>

Feb 24

Claude Code Anniversary + Launches from: Qwen 3.5, Cursor Demos, Cognition Devin 2.2, Inception Mercury 2

>

# 中外最新模型

# OpenAI

- 最新版本：OpenAI o3 (Orion) / GPT-5
- 核心标签：逻辑天花板 · 自主工具链 · 生态壁垒
- 核心突破：
  - o3 (Orion)：专攻“慢思考”。在数学竞赛（AIME）和博士级科学问答（GPQA）中刷新了人类历史记录。它能自主编写并运行代码来验证自己的猜想，大幅减少幻觉。
  - GPT-5：通用能力的集大成者。相比 GPT-4o，在创意写作、多语言理解和复杂指令遵循上有了质的飞跃。
  - Deliberative Alignment (审慎对齐)：在模型思考阶段就引入安全审查，让 AI 更“听话”且安全。



# Google Gemini

- 最新版本：Gemini 3 Pro / 3.1 Flash
- 核心标签：原生多模态 · 200万+窗口 · 动态思考
- 核心突破：
  - Native Multimodal (原生多模态)：搭载最新的 Nano Banana 2 (图像) 和 Veo 3.1 (视频) 模块。它是唯一能实时看懂长视频流、听懂语调情绪并打断对话的模型。
  - Deep Think 模式：引入了动态推理等级，模型会根据问题难度自动决定是“快答”还是“深思”。
  - 无限上下文：支持 2M+ Token 输入，能一次性吃透整个代码库或数小时的会议录像。

# Anthropic Claude

- 最新版本：Claude 4.6 Sonnet / Opus
- 核心标签：混合推理 · 电脑操作 · 代码之神
- 核心突破：
  - Computer Use (电脑操作)：能够像人一样控制鼠标键盘，跨软件（Excel, VS Code, 浏览器）完成复杂任务。
  - Hybrid Reasoning (混合推理)：用户可手动开启“Extended Thinking”模式，让 Claude 在写代码前进行长达数分钟的架构规划。
  - Agent Teams：引入多智能体协作能力，一个指令可唤起“开发”、“测试”、“文档”多个分身协同工作。

# DeepSeek (深度求索)

- 最新版本：DeepSeek-R1 / V3.1
- 核心标签：纯强化学习 · 价格屠夫 · 权重开源
- 核心突破：
  - 纯强化学习 (Pure RL)： R1 证明了不依赖海量人工标注数据，仅靠强化学习自我博弈也能涌现出超越人类的数学/代码能力。
  - 思维链透明化： 它是目前唯一完整公开内部“思考过程 (Chain of Thought)”的顶尖模型，是学术界研究 AI 内部机制的首选。
  - 极致低成本： 依托 MoE (混合专家) 架构，推理成本仅为 OpenAI o1 的 1/30。

# Alibaba Qwen (通义千问)

- 最新版本：Qwen3-Max / Qwen3.5-VL
- 核心标签：双模式思考 · 视觉增强 · 100+语言
- 核心突破：
  - Thinking Mode (思考模式)：Qwen3 引入了可切换的“深度思考”开关，在处理高考数学题或复杂逻辑时表现媲美 DeepSeek-R1。
  - Qwen-VL (视觉语言)：在工业质检、文档图表分析领域处于全球领先地位，支持对图片中的物体进行精准坐标定位 (Grounding)。
  - MoE 架构升级：即使是 Max 版本也采用了高效的 MoE 架构，在保持 SOTA 性能的同时大幅降低了企业 API 调用成本。

# ByteDance Doubao (豆包)

- 最新版本：Doubao 2.0 (Seed 2.0)
- 核心标签：视频生成 · 移动 Agent · 超级应用
- 核心突破：
  - 视频生成原生化：集成了 Seedance 2.0 模型，支持通过自然语言生成电影级的高一致性视频，且运镜控制精准。
  - Agent Era (智能体时代)：重新定义为“能干活的 AI”，重点强化了在手机端的跨 App 操作能力（如一键订票、点外卖）。
  - 日活霸主：依托抖音生态，是目前全球 C 端用户量最大的 AI 应用之一。